

BLM3021 - Algorithm Analysis

"The Algorithm Design Manual"

"An Introduction to the Analysis of Algorithms"

[Syllabus]

Vize	%40	* 2 Vize	%36
Lab	%10	* 3 Lab (2+1)	3 Uygulama
Ödev	%10	* 2 Ödev	%4 (2+1)
Final	%40		

Midterm I (17.11.2025)

Midterm II (15.12.2025)

FO
→ 40 ve üzeri alamaزون
ders başarısız
sayılacak

1. Ders

- Algorithm nedir? vs. vs...

How good is the algorithm?

Does there exist a better algorithm?

• Correctness

• Lower bounds

• Efficiency (Time, Space)

• Optimality

• Simplicity

• Generality

Predict Performance

Reason to Analyze an Algorithm → Compare Algorithms

↘ Avoid Performance Bugs

System Independent

System Dependent

Algorithms

Hardware

OS

Empirical Analysis $T(N)$

Mathematical Models for Running Time

⇒ sum of (cost * frequency for all operations)

Asymptotic Notations

Big-O, Big-Teta, Big Omega ($O(N)$, $\Omega(N^2)$, $\Theta(N)$)

Upper
Bound

Lower
Bound

Tight
Bound

Amortized Analysis

A Large cost of operation is spread out over many operations (amortized)

3 Common Methods

- Aggregate Method
- Accounting Method
- Potential Method

Usage
Examples

- Dynamic Arrays
- Binary Counter
- Stack Operations (Mulltipop)
- Trees (Red-Black Tree)

Amortized Analysis

Find Max

Best Case n
Average Case n
Worst Case n

} $\Theta(n)$
tight bound

```
int sequentialSearch (int n, int a[], int x)
{
    int i = 0;
    while ((i < n) && (a[i] != x))
    {
        i++;
    }
    if (i < n)
        return i;
    else
        return -1;
}
```

Best Case: $O(1)$, Worst Case: $O(n)$

Average Case: $\rightarrow$ probability

$$0 \leq p \leq 1$$

i th position, $\frac{p}{n}$

not in
the array!

$$C_{avg} = \left[1 * \frac{p}{n} + 2 * \frac{p}{n} + \dots + n * \frac{p}{n} \right] + n * (1-p)$$

successful search $\rightarrow p=1$ $\frac{1}{n}(1+2+\dots+n) = \frac{n+1}{2}$
 $O(n)$

$$= \frac{p}{n} [1+2+\dots+n] + n * (1-p)$$

unsuccessful search $p=0$
 $C_{avg} = n * (1-0) \rightarrow O(n)$

$$\sum_{i=1}^n 1 = n$$

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i^2 = 1^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} = \frac{n^3}{3}$$

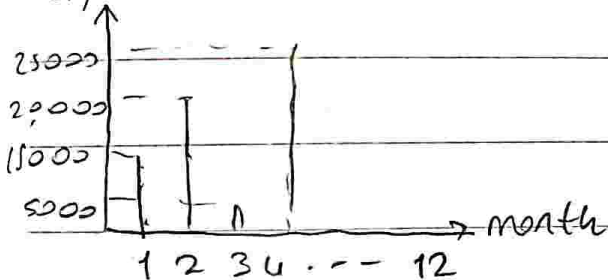
$$\sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \dots + \frac{1}{n} = \ln(n)$$

Amortized Analysis $\neq$ Average Case Analysis

→ worst case analysis

(1) The Aggregate Method, Brute Force

expenses



$$\frac{1}{12} * \sum_{i=1}^{12} \text{expense}[i]$$

k-bit binary counter:

8bit

	7	6	5	4	3	2	1	0	cost	total cost
0	0	0	0	0	0	0	0	0	0	0
1					0	0	0	1	1	1
2					0	0	1	0	2	3
3					0	0	1	1	3	4
4					0	1	0	0	3	7
5					0	1	0	1	4	8
6										
7										
...										
16										

for n increments:

bit 0 flips n times

bit 1 flips n/2 times

" 2 " n/4 "

" J " n/2^j "

total flipped bits

$$\sum_{j=0}^{k-1} \left\lceil \frac{n}{2^j} \right\rceil \rightarrow n \left(1 + \frac{1}{2} + \dots + \frac{1}{2^{\log n}} \right)$$

(31)

$$\underbrace{n + \frac{n}{2} + \dots + \frac{n}{2^j}}_{F(n)} \leq c * (g(n))$$

$$F(n) \leq 2 * n \Rightarrow O(n)$$

Amortized Analysis ile k-bit counter karmaşıklığı $O(n)$ bulundu.
(per increment: $\frac{O(n)}{n} = O(1)$)

WORST CASE hesabı ile:

0111
1000

k: bit sayısı $\Rightarrow$ worst case $= O(k)$ $\left(k = \log_2 n \right)$

$\rightarrow$ per increment
 $\rightarrow$ 1 increment
n defa değişim: $O(n * \log_2 n)$

Böyle değişimli algoritmalarda amortize analiz daha gerçekçi bir sonuç vermektedir.

Kullanım Alanları

Augment Stack

push(S, x) $O(1)$

pop(S) $O(1)$

multiPop(S, k): pops the min(k, |S|)

$\hookrightarrow$ if $k \leq |S| \rightarrow k$ $O(n)$ worst case

max num. of pops: n (stack size)

$\hookrightarrow O(n)$ otherwise |S|

Amortized cost per operation:

of pops OR # of multiple pops: $\leq$ # of pushes
 $\leq O(n)$

Amortized Analysis $\frac{O(n)}{n} = O(1)$

Aggregate Method

- ① Identify all operations (total of n)
- ② Determine the actual cost of each operation
- ③ Sum all

Accounting Method

Assign amortized cost to each operation

Each increment: 1 coin

$0 \rightarrow 1$: 1 coin

$1 \rightarrow 0$: pay 1 coin

- ① At most one $0 \rightarrow 1$ per increment
 n increment : $\#(0 \rightarrow 1) : n$

- ② Every $(1 \rightarrow 0)$ is paired with previous $(0 \rightarrow 1)$
 $\#(1 \rightarrow 0) \leq \#(0 \rightarrow 1)$

- ③ $\# \text{ flips} : \#(0 \rightarrow 1) + \#(1 \rightarrow 0) \leq \underbrace{\#(0 \rightarrow 1)}_n + \underbrace{\#(0 \rightarrow 1)}_n \leq 2n$

Total Cost: $2n$ Amortized Cost: $\frac{n}{n} = O(1)$

Binary	Bits Flipped	Cost
000	0	0
001	1	1 gain
010	2	1 gain
011	1	2 gain
100	3 $\begin{smallmatrix} 0 \rightarrow 1: 1 \\ 1 \rightarrow 0: 2 \end{smallmatrix}$	1 gain

Hiç negatif düşmez!

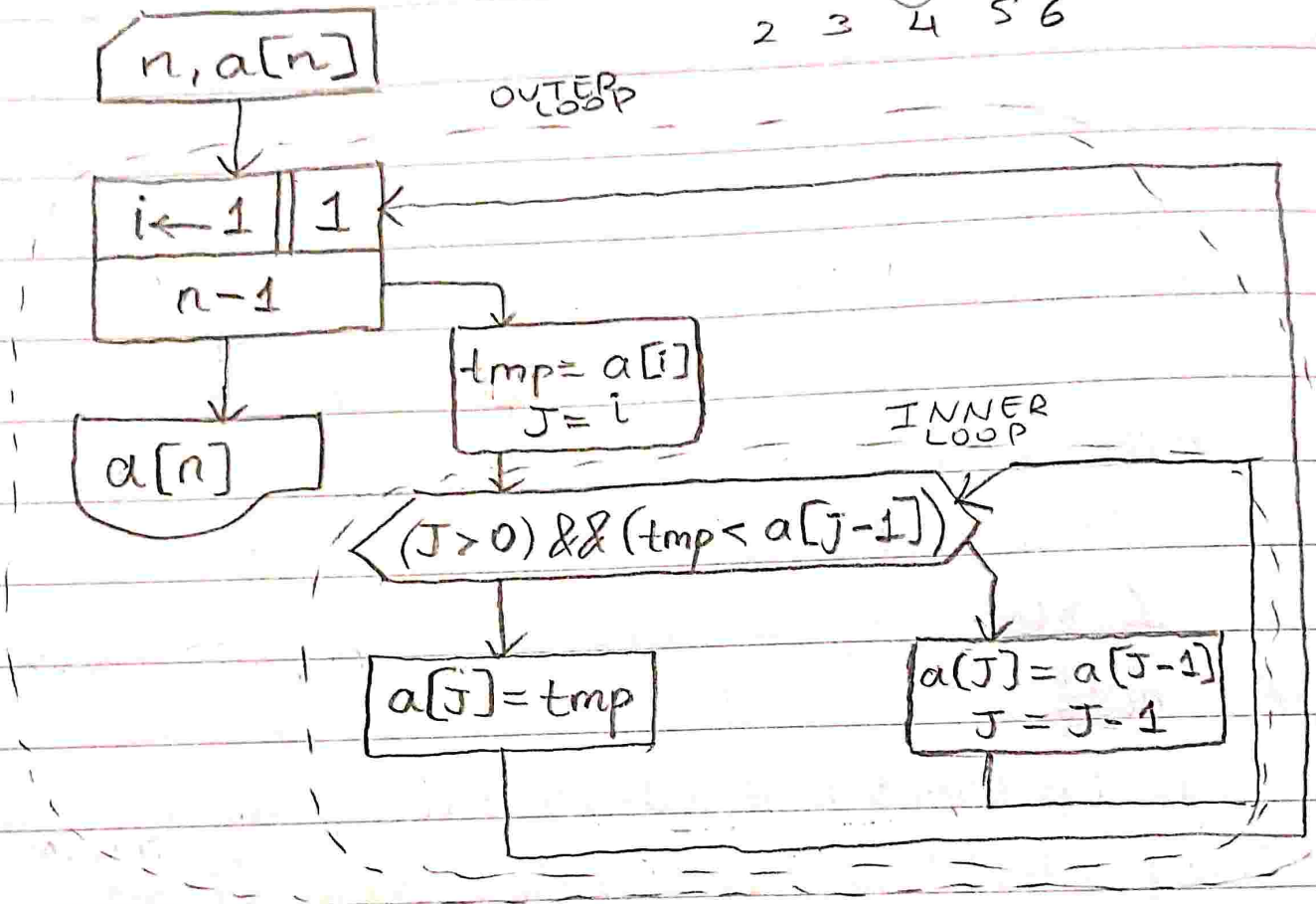
after n increment actual cost ≤ 2

Analysis of Recursive and Non-Recursive Algorithms

(Analysis of)
non-recursive algorithms

① Insertion Sort

0	1	2	3	4
4	2	3	5	6
2	4	3	5	6
2	3	4	5	6



⇒ Worst Case Büyük → Küçük
9 7 5 4 2

$$\sum_{i=1}^{n-1} i = \frac{n(n+1)}{2} = \frac{n^2}{2} \in O(n^2), \quad C_{\text{worst}} \in \Theta(n^2) \checkmark$$

tight bound

⇒ Best Case Küçük → Büyük

$$\sum_{i=1}^{n-1} 1 = n-1$$

1 2 3 4 5

$$C_{\text{best}} \in O(n), \quad C_{\text{best}} \in \Theta(n)$$

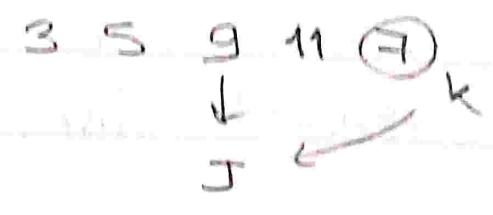
⇒ Average Case

Expected cost of random input

Outer loop is fine, hard part is inner loop because WHILE

inner loop

⇒ element is at the k th position
it winds up at position J



Final position: J

$(K - J + 1 \text{ comparison})$

on average: $\frac{1}{K} \sum_{J=1}^K (K - J + 1) = \frac{1}{2} \left(K^2 - \frac{K(K+1)}{2} + K \right)$

of comparison = $\frac{2K^2 - K^2 - K + 2K}{2K} = \frac{K^2 + K}{2K}$

$= \frac{K+1}{2}$ → inner loop

outer loop

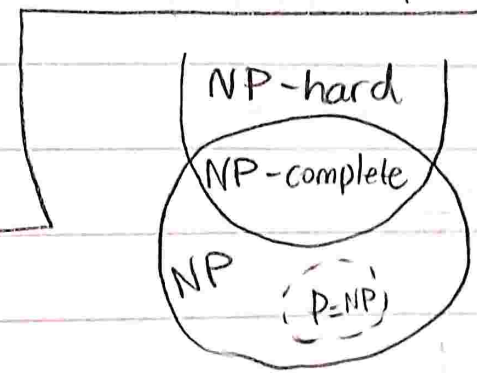
$\sum_{K=1}^{N-1} \left(\frac{K+1}{2} \right) = \frac{1}{2} \left(\frac{(N+1)(N+2)}{2} \right) = \frac{N^2 + 3N + 2}{4}$ { Coverage $\in O(N^2)$
 $\in \Theta(N^2)$

Insertion sort $\in O(n^2)$

~~$\in \Theta(n^2)$~~

$\in \Omega(n)$ (Best case)

(MEK BURAYI İŞLEMEDİ!) →



(son hafta işledi)

Problem Sınıfı

P (polynomial time)

NP (non-det. poly. time)

NP-hard (atleast as hard as the hardest NP problem)

NP-complete (both NP and NP-hard, polynomially reducible to D)

Bu sınıflandırmalar problemin çözülebilirliği ve çözme süresi ile ilişkilidir.

(Analysis of) recursive functions

Recurrence Relation:

$$f(n) = \begin{cases} 1 & n=0 \\ n * f(n-1) & n>0 \end{cases} \quad (\text{faktöriyel algoritması})$$

$$M(n) = M(n-1) + 1 \quad \leftarrow 1. \text{ adımda } 1 \text{ qarpma var!!!}$$

$$M(0) = 0 \quad n=0$$

GÖZÜM YOLLARI

① Backward Substitution

$$\begin{aligned} M(n) &= M(n-1) + 1 \\ &= \underbrace{M(n-2) + 1}_{\downarrow} + 1 \\ &= \dots \end{aligned}$$

$$M(n) = M(n-i) + i \quad \begin{matrix} n-i=0 \\ n=i \end{matrix} \rightarrow M(n) = \underbrace{M(0)}_0 + n$$

$$\begin{aligned} M(n) &\in O(n) \quad \leftarrow M(n) = n \\ &\in \Theta(n) \\ &\approx \Omega(n) \end{aligned}$$

② Forward Substitution

<u>n</u>	<u>m(n)</u>
0	0
1	1
2	2
...	
n	$m(n-1) + 1 = n$

③ Master Theorem

Running Time (General ^{Divide} Conquer Recurrence R.)

$$T(n) : a * T(n/b) + f(n)$$

of subproblems $\downarrow$ size of each subproblem

void mergeSort (int a[], int p, int r) → A[p...r]
↓ ↓
0 n-1: initial

```

{
    if (p < r)
    {
        q = (p + r) / 2;
        mergeSort (A, p, q);
        mergeSort (A, q + 1, r);
        merge (A, p, q, r);
    }
}

```

$$T(n)_{\text{Merge sort}} = \begin{cases} 1 & \text{if } n=1 \\ 2 * T(n/2) + n & \text{if } n > 1 \end{cases}$$

↓
C_{merge}

MASTER THEOREM $T(n) = a * T(n/b) + f(n)$

If $f(n) \in \Theta(n^d)$ where $d \geq 0$

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } d < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

↓
(O veya Ω de olur)

Can't be used: $T(n) = \sin n$, $F(n) = 2^n$, $a = 2 * n \dots$

$$T(n)_{\text{merge sort}} = \begin{cases} 1 & , n=1 \\ 2T(n/2) + n & , n>1 \end{cases}$$

Backwards Substitution

$$T(n) = 2 * T(n/2) + n$$

$$T(n) = 2 * [2 * T(n/4) + n/2] + n = 4T(n/4) + 2n$$

$$T(n) = 2 * [2 * [2T(n/8) + n/4] + n/2] + n = 8T(n/8) + 3n$$

$$8T(n/8) + 3n \quad n = 2^i \quad i = \log_2 n$$

$$\begin{matrix} \downarrow & \downarrow & \downarrow & \downarrow \\ 2^i & 2^i & 2^i & i \end{matrix}$$

$$n * \underbrace{T(1)}_1 + \log_2 n * n$$

$$2^i * T(2^i / 2^i) + i * 2^i$$

$$T(n) \in O(n \log n)$$

Master Theorem

$$a=2, b=2, d=1$$

$$T(n) \in \Theta(n^1 * \log n)$$

python

dynamic array

amortized analysis

list.append

list.pop

list.insert

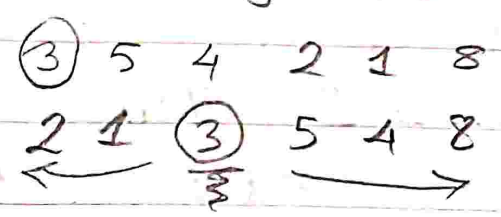
haffaya arastir.

Divide and Conquer

Quick sort, Merge sort

$O(n^2) \rightarrow O(n \log n)$
sort algorithms

- ① Divide ② Conquer ③ Combine



Decrease and Conquer

Binary Search, Insertion Sort, Topological Sort

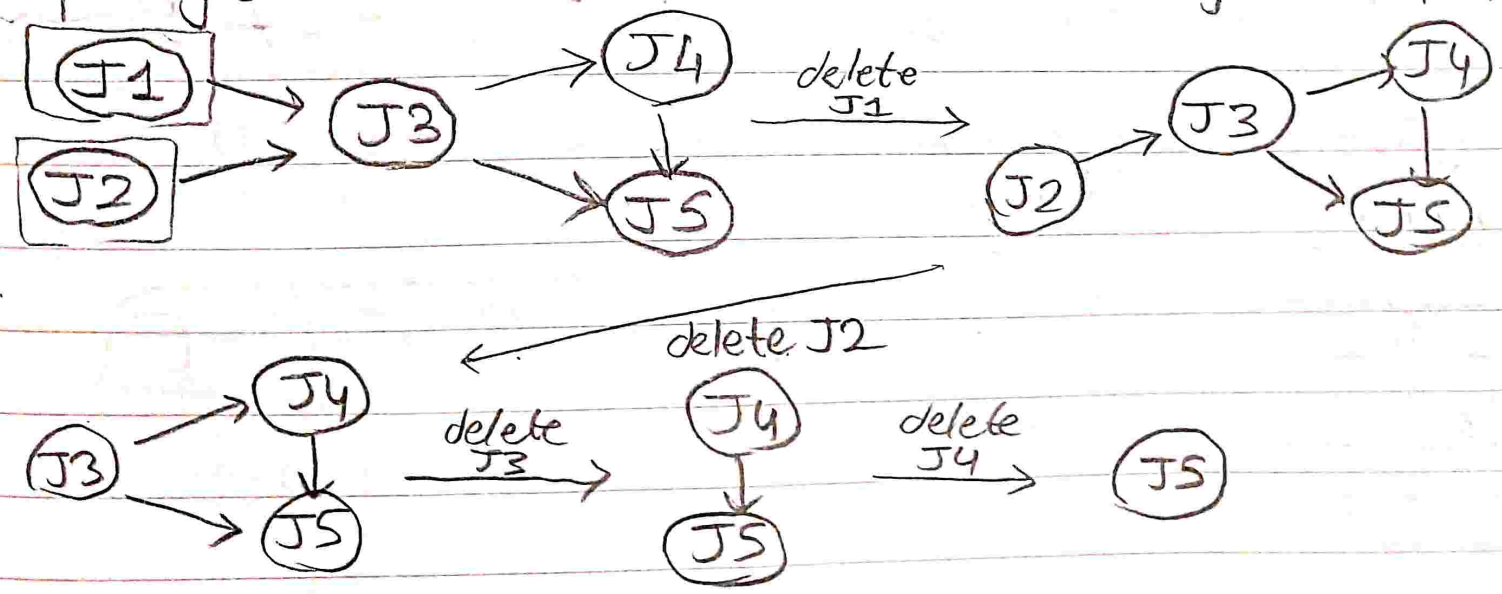
3 yolu var

- ① Decrease by a constant (usually by one) Topological Sort
Insertion Sort

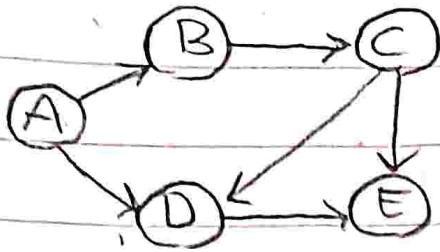
$f(n) = a^n = a^{n-1} * a$

Topological Sort

(Graph must be Directed Acyclic Graph)



- J1 - J2 - J3 - J4 - J5
J2 - J1 - J3 - J4 - J5



Indegree

0 A → [B] → [D] → NULL

1 B → [C] → NULL

1 C → [D] → [E] → NULL

2 D → [E] → NULL

2 E → NULL

0 F → NULL

↓
(Bir listeden elemana ulaşabiliyorsa Indegree'sine +1 eklenir)

Topological Sort

Her seferinde baştan 0

olan elemanı seç ve listeden sıradaki elemanların Indegree değerini 1 azalt

A → B → D ⇒ B-1 → 0, D-1 → 1 Sırada B (0)

B → C ⇒ C-1 → 0 Sırada C

C → D → E ⇒ D-1 → 0, E-1 → 1 Sırada D

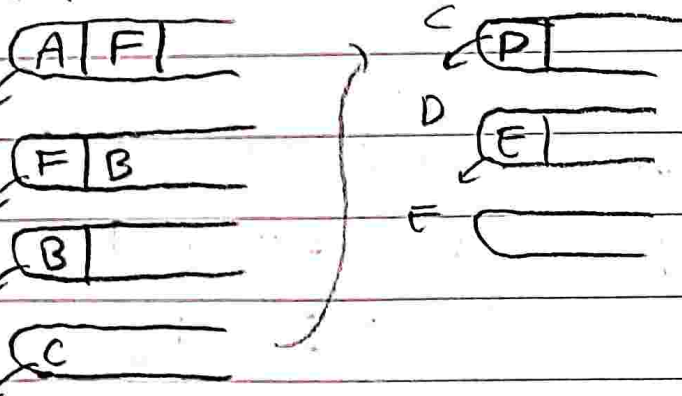
D → E ⇒ E-1 → 0 Sırada E

E → NULL Sırada F

F → NULL

Topological Sort ⇒ **A-B-C-D-E-F**

QUEUE → Indegree değeri 0 olanları kuyruğa ekle



A-F-B-C-D-E

Topological Sort

- ① Initialize $O(|E|)$ → store each vertex's indegree as an array
- ② Initialize queue with all indegree zero vertices $O(|V|)$

(1) indegree

A	B	C	D	E	F
0	1	1	2	2	0

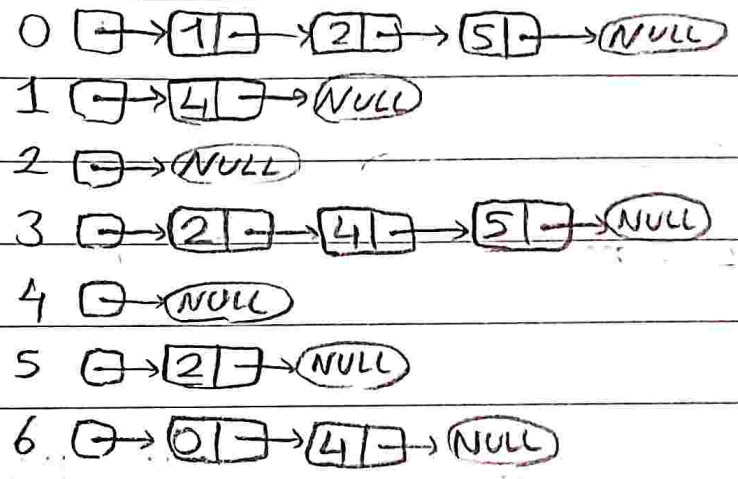
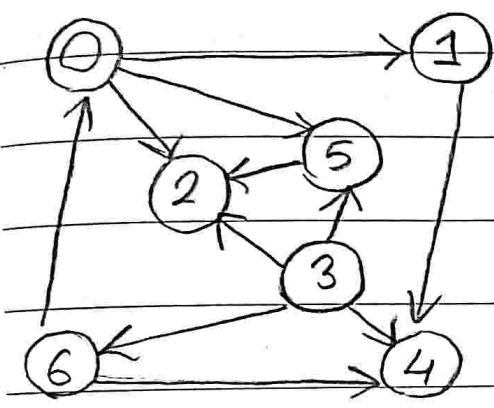
 (2)

A	F				
---	---	--	--	--	--

 queue

- ③ While there are vertices remaining in the queue
 - Dequeue and print $O(|V|)$
 - Reduce indegree of all vertices adjacent to it by 1
 - Enqueue adjacent vertices with indegree zero $O(|E|)$

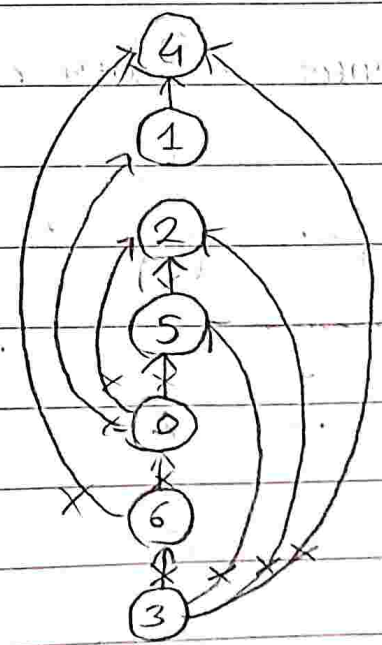
→ $O(|V| + |E|)$ → Linear complexity



DFS

dfs(0)
 dfs(1)
 dfs(4)
 4 done
 1 done
 dfs(2)
 2 done
 dfs(5)
 check 2
 5 done
 0 done
 check 1
 check 2

dfs(3)
 check 2
 check 4
 check 5
 dfs(6)
 check 0
 check 4
 6 done
 3 done
 check 4
 check 5
 check 6
 [DONE]



REVERSE ORDER TOPOLOGICAL ORDER
 3 6 0 5 2 1 4 3 6 0 1 4 5 2

Decrease and Conquer

② Decrease by a constant factor ^{Backwards Substitution} (Binary Search)

Comparison: $C_w(n) = \begin{cases} C_w(n/2) + 1 & \text{for } n > 1 \\ C_w(1) = 1 \end{cases}$

Backwards Substitution

$$\begin{aligned} C_w(n) &= C_w(n/2) + 1 \\ &= C_w(n/4) + 1 + 1 \\ &= C_w(n/8) + 1 + 1 + 1 \end{aligned}$$

$$C_w(n) = C_w(n/2^i) + i$$

$n = 2^i$

for $n = 2^i$

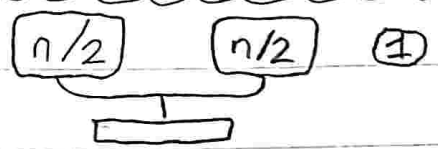
$$C_w(n) = C_w(2^i/2^i) + i$$

$$= 1 + \log_2 n$$

$n = 2^i$
 $i = \log_2 n$

$$\begin{aligned} C_{\text{worst}} &\in O(\log_2 n) \\ C_{\text{BS}} &\in O(\log_2 n) \\ C_{\text{worst}} &\in O(\log_2 n) \end{aligned}$$

Take Coin Problem



7 paranın sahte olanını bulma
(Decrease by a constant factor)

③ Variable Size Decrease Algorithms

finding the k^{th} smallest element!

linear sort with counting

0	1	2	3	4	
7	1	2	4	8	A
1	1	1	1	1	Count
2		2	2	2	
3		3	3	3	
4			4	4	

Transform and Conquer

How do you find duplicates?

③ 4 5 ③ 8 2 4

Brute Force

$O(n^2)$
Diziyi tarayıp
Jeni diziyeye aktarma

Better

Sort $[O(n \log n)]$
Compare $[O(n)]$

two stage process

- ① transformation stage for easier and faster solution
- ② conquering stage

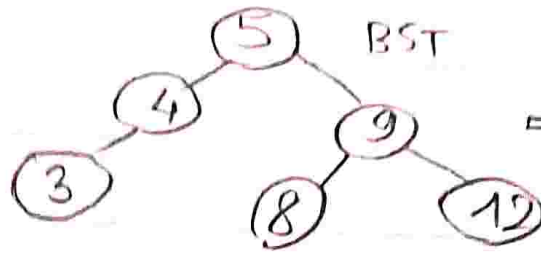
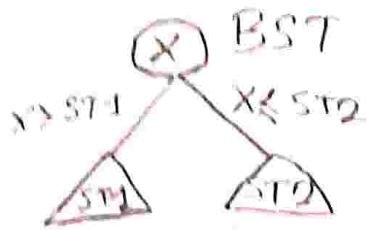
Presorting

- ① unordered list $\rightarrow$ ordered list
- ② conquer: linear scan

Transform and Conquer

Balanced Search Trees

(AVL-trees, Red-Black tree)



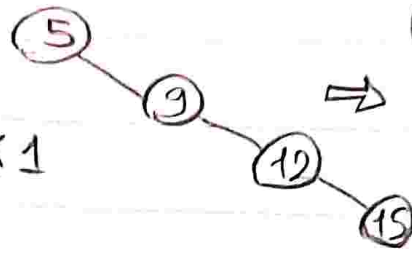
Search Complexity:

Balanced Tree Worst Case: $O(\log_2 n)$

Balanced BST:

$|\text{height}(\text{leftST}) - \text{height}(\text{rightST})| \leq 1$

(ST: Subtree)



Unbalanced

BST Worst Case: $O(n)$

We can transform it

AVL tree (Ağacın adı onu bulan insanın adından gelmektedir.)

When searches are MORE FREQUENT than INSERTION and DELETION

Balance Factor:

Each node in an AVL tree:

$BF = |\text{height}(\text{leftST}) - \text{height}(\text{rightST})| \leq 1$ OLMALI

insert: 5, 6, 8

0(5) BF=0

(5) -1 BF=1
(6) 0 BF=0

(5) -2 BF=2
(6) -1
(8) 0

(5) -2
(6) 0
(8) 0

AVL BST Insertion

① BST Insertion

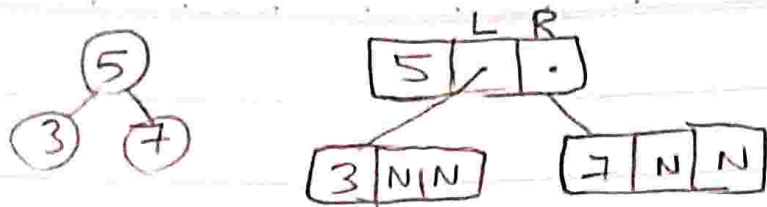
② Rebalancing

1. Insert the key as in normal BST

2. Update the height of every node

3. Compute BF of each node

4. If $BF > +1$ OR -1 perform ROTATIONS to restore Balance!



```

struct Node
{
    int key;
    struct Node *left;
    struct Node *right;
    int height;
}

```

```

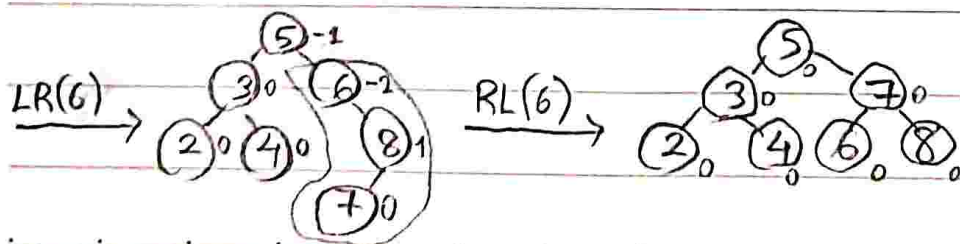
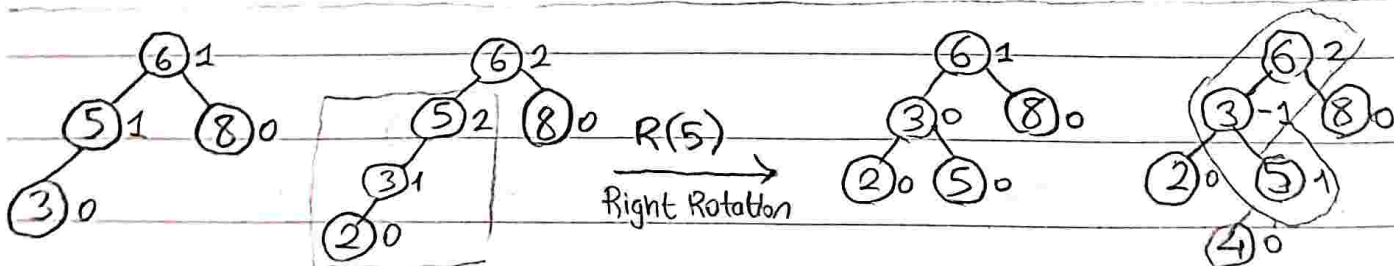
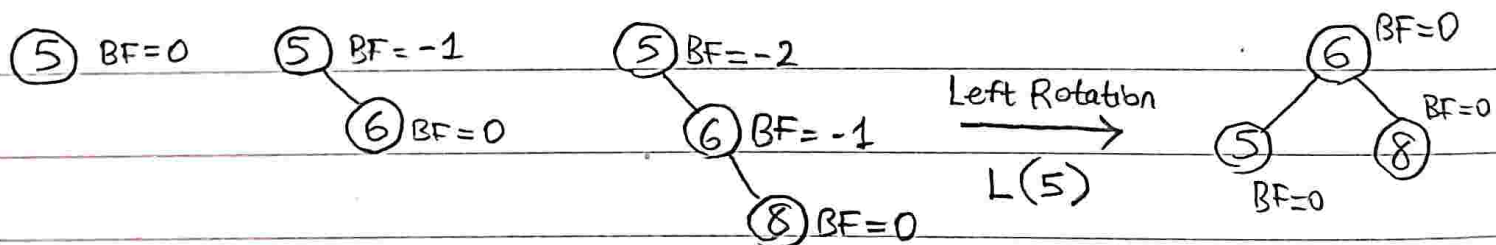
int getHeight(struct Node *N)
{
    if (N == NULL)
        return 0;
    return N->height;
}

```

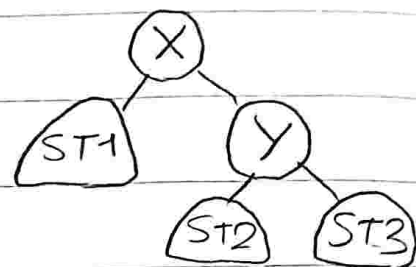
Example

Construct AVL Tree for:

5, 6, 8, 3, 2, 4, 7



LEFT ROTATION (counter-clockwise)



RIGHT ROTATION (clockwise)

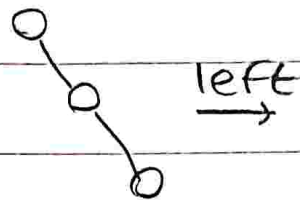
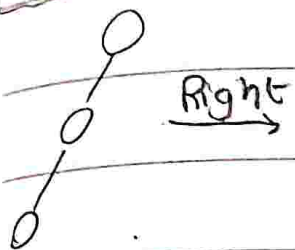
Rotations in AVL Tree

- ① Left Rotation
- ② Right Rotation
- ③ LR Rotation
 - ① Left R.
 - ② Right R.
- ④ RL Rotation
 - ① Right R.
 - ② Left R.

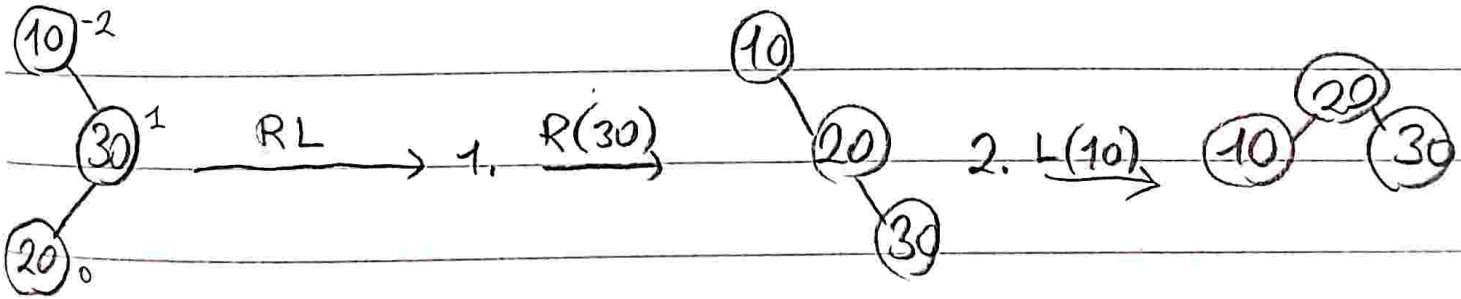
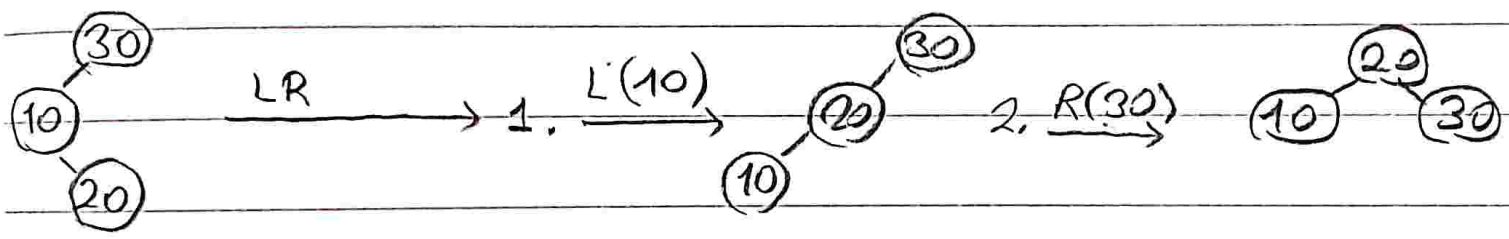
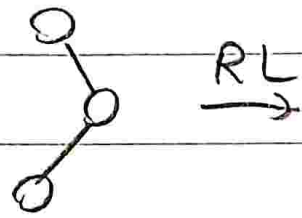
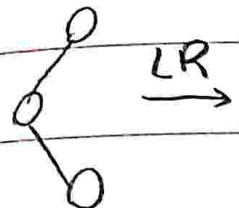
```

struct Node *rightRotate(struct Node *y)
{
    struct Node *x = y->left;
    struct Node *t2 = x->right;
    x->right = y;
    y->left = t2;
    return x;
}

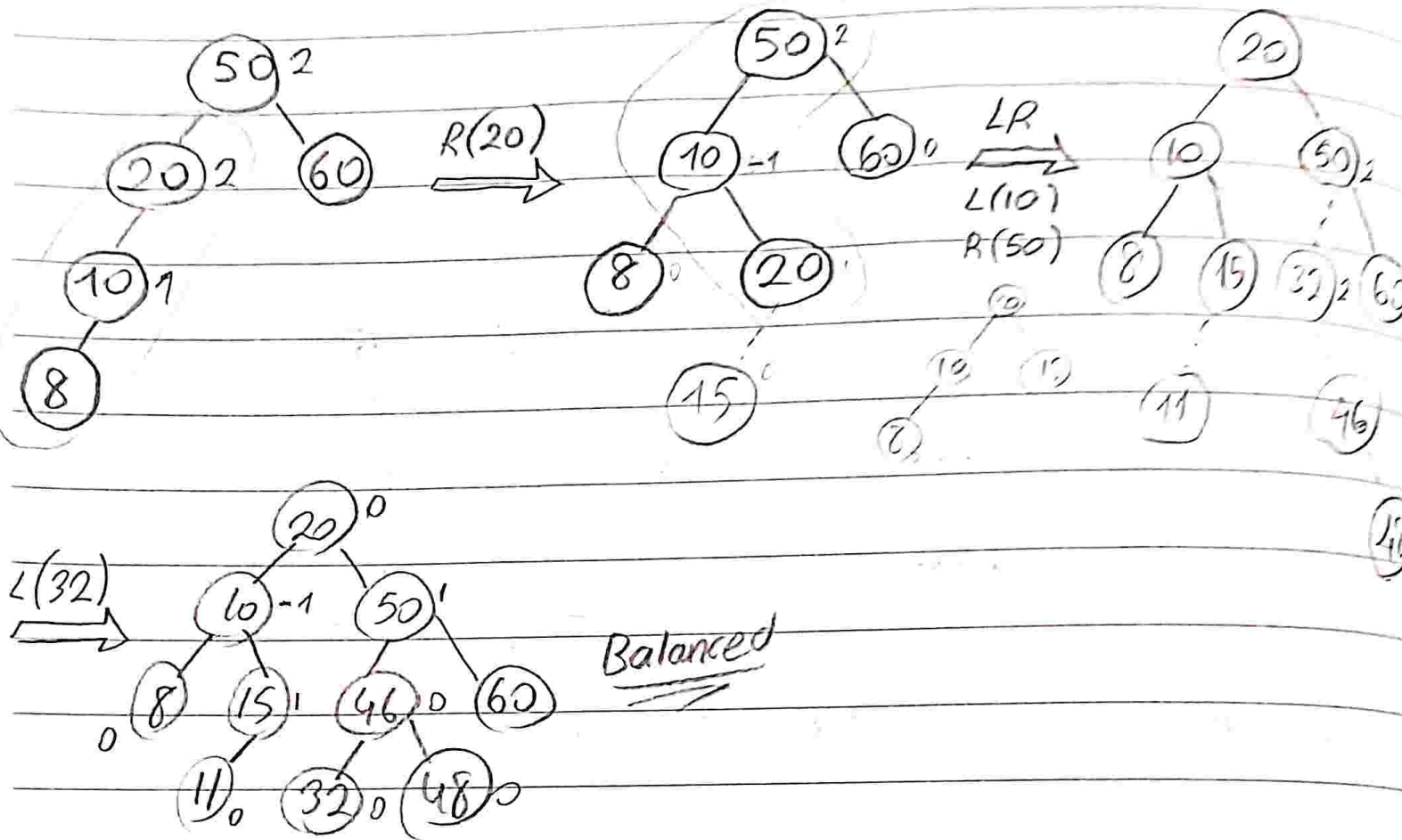
```



Genel Kural



50, 20, 60, 10, 8, 15, 32, 46, 11, 48



[Kth smallest number - pivot approach] UYGULAMA UYGULAMA

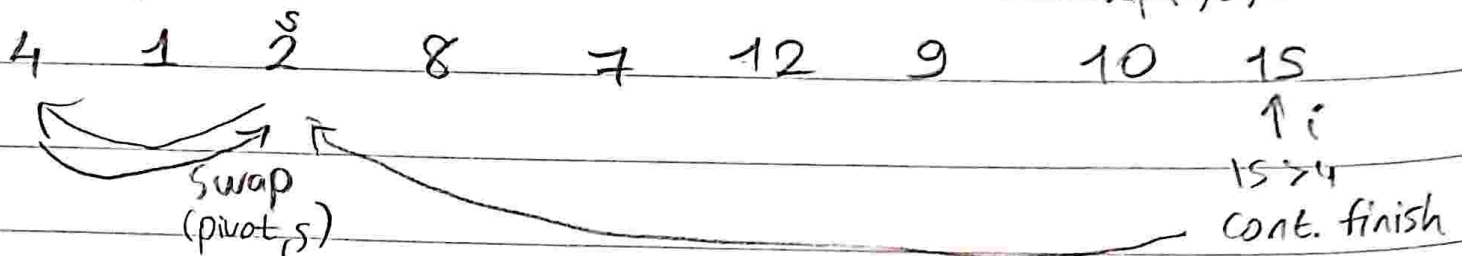
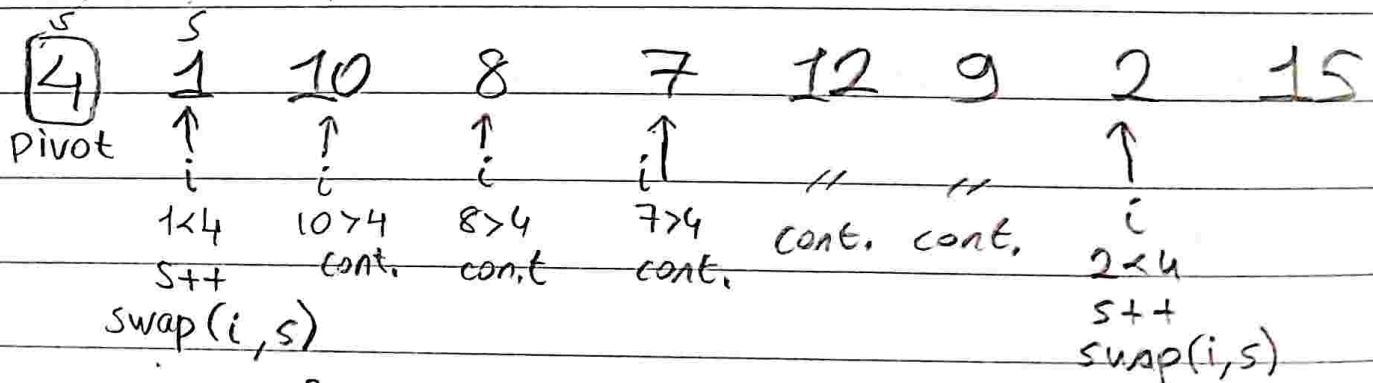


Quick Select

$\leq P$ (P) $\geq P$

2 $\xrightarrow{k=3}$

$k=5?$



2 1 4 8 7 12 9 10 15
 0 1 2 3 4 5 6 7 8

$s = ? k-1$ ($k=5$, 5. en küçük elemanı oruyoruz.)

$2 < 4$

2 1 4 8 7 12 9 10 15
 pivot $\uparrow_i$ $\uparrow_i$ $\uparrow$ $\uparrow$ $\uparrow_i$
 $7 < 8$ $12 > 8$ // // //
 s++ cont cont cont cont
 swap finish

2 1 4 7 8 12 9 10 15
 0 1 2 3 4 5 6 7 8
 swap(pivot, s)

$s = ? k-1$

$4 = 4 \rightarrow 8 = k^{\text{th}}$ smallest number
 ($k=5$)

Worst Case $\rightarrow$ Pivot her işlemde aynı kalırsa $O(n^2)$

Average Case $\rightarrow O(n)$

Median of Two Sorted Arrays

$$A = [a_1, a_2, \dots, a_n]$$

$$B = [b_1, b_2, \dots, b_m]$$

Bu iki diziyi birleştirmeden
birleşik dizinin ortanca değerini bulma

$$A = 1, 4, 7$$

$$B = 2, 3, 5, 8$$

$$> 1, 2, 3, (4), 5, 7, 8 \quad ?$$

$$A = \begin{array}{|c|c|c|c|c|} \hline 1 & 12 & 15 & 26 & 38 \\ \hline \end{array} \quad n=5$$

$$B = \begin{array}{|c|c|c|c|c|c|} \hline 2 & 13 & 17 & 30 & 45 & 60 \\ \hline \end{array} \quad m=6$$

$$L = \frac{m+n+1}{2} = 6$$

(0, 5)
low high

$$\text{mid}_1 = \frac{\text{low} + \text{high}}{2} = 2, \quad \text{mid}_2 = L - \text{mid}_1 = 4$$

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 12 & 15 & 26 & 38 \\ \hline \end{array}$$

$l_1 \quad \text{mid}_1 \quad r_1$

$$l_1 < r_2 \quad \checkmark$$

$$\begin{array}{|c|c|c|c|c|c|} \hline 2 & 13 & 17 & 30 & 45 & 60 \\ \hline \end{array}$$

$l_2 \quad \text{mid}_2 \quad r_2$

$$l_2 < r_1 \quad \times \rightarrow \text{mid}_1 \text{ sağa kaymalı}$$

$$\text{low} = \text{mid} + 1 = 3$$

$$\text{high} = 5$$

$$\text{mid}_1 = \frac{3+5}{2} = 4 \quad \text{mid}_2 = 6 - 4 = 2$$

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 12 & 15 & 26 & 38 \\ \hline \end{array}$$

$l_1 \quad \text{mid}_1 \quad r_1$

$$\begin{array}{|c|c|c|c|c|c|} \hline 2 & 13 & 17 & 30 & 45 & 60 \\ \hline \end{array}$$

$l_2 \quad \text{mid}_2 \quad r_2$

$$l_1 < r_2 \quad \times \rightarrow \text{low} = 3$$

$$l_2 < r_1 \quad \checkmark$$

$$\text{high} = \text{mid}_1 - 1 = 4$$

$$\text{mid}_1 = 3$$

$$\text{mid}_2 = 3$$

$$\begin{array}{|c|c|c|c|c|} \hline 1 & 12 & 15 & 26 & 38 \\ \hline \end{array}$$

$l_1 \quad \text{mid}_1 \quad r_1$

$$\begin{array}{|c|c|c|c|c|c|} \hline 2 & 13 & 17 & 30 & 45 & 60 \\ \hline \end{array}$$

$l_2 \quad \text{mid}_2 \quad r_2$

$$l_1 < r_2 \quad \checkmark$$

$$l_2 < r_1 \quad \checkmark$$

MEDYAN:

$$\max(A[\text{mid}_1], B[\text{mid}_2])$$

$$= 17$$

Dynamic Programming → planning

subproblems are **DEPENDENT** ⇒ recursion kullanılmamalı

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

1 1 2 3 5 8

```
int fibonacci(int n)
```

```
{
```

```
    if (n == 0)
```

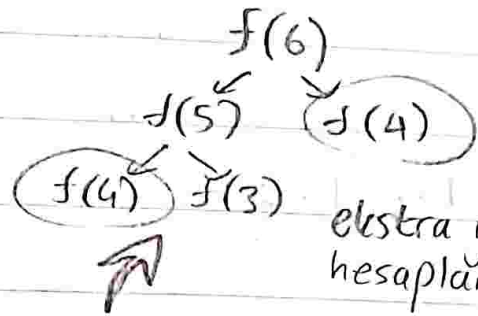
```
        return 0;
```

```
    if (n == 1)
```

```
        return 1;
```

```
    else return (fibonacci(n-1) + fibonacci(n-2));
```

```
}
```



ekstra yeniden
hesaplama var...

verimli değil

1. Define subproblems
2. Write the recurrence relation
3. Solve the **BASE CASES**
4. **RECORD SOLUTIONS** in a **TABLE**

→ array, matrix etc.

$$\text{fibonacci}[n] = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fibonacci}[n-1] + \text{fibonacci}[n-2] & \text{if } n \geq 2 \end{cases}$$

fib[0] = 0;

fib[1] = 1;

for (i=2; i<n; i++)

fib[i] = fib[i-1] + fib[i-2];

minimum coin problem

you are given a set of coins: $\{1, 2, 5\}$

A target amount is: 11 TL

$$1+1+1 \dots = (11)$$

$$\underbrace{2+2+\dots+2}_{\times 5} + 1 = (6)$$

$$5+5+1 = (3)$$

A: target amount

$dp[x]$: minimum coins needed to make amount x $dp[11]=3$

initial $\{dp[0]=0$ (zero coin): base case

$$dp[i]=\infty \quad i=1 \text{ to } A$$

MAXINT

(örnek için 11 TL)

$$x=1 \rightarrow \text{try coin 1: } 1 + dp[\underbrace{1-1}_{\substack{\downarrow \\ 1 \text{ TL} \\ (c)}}] = 1 + 0 = 1$$

$$\begin{aligned} x=2 \rightarrow \text{try coin 1: } 1 + dp[\underbrace{2-1}_1] &= 2 \text{ coin} \\ \text{try coin 2: } 1 + dp[\underbrace{2-2}_2] &= 1 \text{ coin} \end{aligned} \quad \left. \vphantom{\begin{aligned} x=2 \rightarrow \text{try coin 1: } 1 + dp[\underbrace{2-1}_1] &= 2 \text{ coin} \\ \text{try coin 2: } 1 + dp[\underbrace{2-2}_2] &= 1 \text{ coin} \end{aligned}} \right\} \min(2, 1) = 1 //$$

$$\begin{aligned} x=3 \rightarrow \text{try coin 1: } 1 + dp[\underbrace{3-1}_2] &= 1 + 1 = 2 \text{ coin} \\ \text{try coin 2: } 1 + dp[\underbrace{3-2}_1] &= 1 + 1 = 2 \text{ coin} \end{aligned} \quad \left. \vphantom{\begin{aligned} x=3 \rightarrow \text{try coin 1: } 1 + dp[\underbrace{3-1}_2] &= 1 + 1 = 2 \text{ coin} \\ \text{try coin 2: } 1 + dp[\underbrace{3-2}_1] &= 1 + 1 = 2 \text{ coin} \end{aligned}} \right\} \min(2, 2) = 2 //$$

$$\begin{aligned} x=4 \rightarrow \text{try coin 1: } 1 + dp[4-1] &= 1 + 2 = 3 \text{ coin} \\ \text{try coin 2: } 1 + dp[4-2] &= 1 + 1 = 2 \text{ coin} \end{aligned} \quad \left. \vphantom{\begin{aligned} x=4 \rightarrow \text{try coin 1: } 1 + dp[4-1] &= 1 + 2 = 3 \text{ coin} \\ \text{try coin 2: } 1 + dp[4-2] &= 1 + 1 = 2 \text{ coin} \end{aligned}} \right\} 2 //$$

Recurrence Relation:

initialize:

$$dp[x] = \begin{cases} 0 & x=0 \\ \min(1 + dp[x-c]) & x>0 \\ c \in \text{coins}, x-c \geq 0, \text{ for all coins } c \end{cases}$$

$$dp[x] = \begin{cases} 0 & x=0 \\ \infty & x>0 \end{cases}$$

RUNNING TIME:

$$O(A \times \text{number of coins})$$

target amount

$$\text{örnek: } dp[6] = 1 + dp[6-1] = 2$$

$$1 + dp[6-2] = 3$$

$$1 + dp[6-5] = 2$$

2 //

Brute Force:

2ⁿ // 0-1 Knapsack Problem W: maximum weight, # of items
 ↓ abandon → select (can't repeat items) value ← weight
 Gerçek zamanda çözülmez. (exhaustive search)

1 ≤ i ≤ n : weights : w₁, ..., w_n maximize ∑ v_i subject to W
 : value : v₁, ..., v_n ∑ w_i ≤ W

weight
 item P[i, j] : maximum profit at [i, j] (örnek P[2, 3] 2. item 3. kg için max karzang)
 item weight value

1 → 2 12
 2 → 1 10
 3 → 3 20
 4 → 2 15

case 1: thief takes item i

$$P[i, w] = P[i-1, w-w_i] + v_i$$

case 2: thief does NOT takes item i

$$P[i, w] = P[i-1, w]$$

W = 5kg →

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

BEFORE item i

4. itemdayken 5kg ağırlıkta karzang (P[4, 5])

3. item 3kg drumu

1, 2 ve 4 alındı

$$\max \left\{ \underbrace{P[3, 5-2] + 15}_{\text{taken}}, \underbrace{P[3, 5]}_{\text{not taken}} \right\} = 37 \text{ TL}$$

$$P[i, w] = \begin{cases} P[0, j] = 0, P[i, 0] = 0 \text{ if } i=0, j=0 : \text{initialize!} \\ P[i-1, w] & \text{if } w_i > w \text{ (NOT TAKEN)} \\ \max \left\{ \underbrace{(v_i + P[i-1, w-w_i])}_{\text{TAKEN!}}, \underbrace{P[i-1, w]}_{\text{NOT TAKEN!}} \right\} & \text{otherwise} \end{cases}$$

matrisle çözüldüğü için 2ⁿ → O(n * w) ✓

hangi eşyaların alındığını bulmak için geri gitme:

START at $P[n, w]$

(n : # of items)

when you go straight up: NOT TAKEN! 

$(P[i, w] == P[i-1, w])$

when you go LEFT up: item has TAKEN! 

RED BLACK TREE (RBT)

Another self-balancing BST (search, insert, delete $\rightarrow O(\log n)$)

→ Every node is BLACK or RED

→ The root is BLACK

→ All null nodes are considered BLACK.

→ A RED node does not have a RED child

→ Every path from a given node to any of its descendant null leaves goes through the same number of BLACK nodes.

Edit Distance

spell checking

graffe → graph, graft, grail, giraffe

Edit Distance

kitten → sitting

maths → maths

→ delete → insert

distance: 3

→ Insert

→ Delete (remove)

→ Replace (modify)

each operation costs 1 point

karga
↓ replace
karga

Edit Distance / Levenshtein Distance:

minimum number of editing operations needed to transform a string into another string.

word1_N → word2_m

also used in computational biology

ACGGTA
TACGTA

word 2

w				
0				
r				
d				
1				

word1 [0...i-1]
word2 [0...j-1]

ED[i][j]: word1[i] } minimum number of edits for
word2[j] } first i char of word1 and
first j char of word2

① Initialization

ED[i][0] = i (no word2): i deletions

ED[0][j] = j (no word1): j insertions

word1 word2
"MA" " "

		A	R	T	S
	0	1	2	3	4
M	1				
A	2				
T	3				
H	4				
S	5				

a) For Deletion

word1 → word2

KAR A → KAA

→ delete

word1 [0...i-1] } compare
word2 [0...j-1]

If we delete LAST CHAR of word1, we need to match

X[0...i-1] → Y[0...j-1]

word1 [i-1] = "R"

total cost:

ED[i][j] = ED[i-1][j] + 1

b.) For Insertion

$KAS \rightarrow KAS[A]_{insert}$

$word1[0...i] \rightarrow word2[0...j-1]$

$$ED[i][j] = ED[i][j-1] + 1 \rightarrow \text{cost}$$

$$ED[i][j] = \begin{cases} \text{Insert: } ED[i][j-1] + 1 \leftarrow \text{Extending word2 by one extra character} \\ \text{Delete: } ED[i-1][j] + 1 \uparrow \\ \text{Replace: } ED[i-1][j-1] + 1 \nwarrow \end{cases}$$

when $word1[i][j] \neq word2[i][j]$

② Recurrence Relation:

$$ED[i, j] = \begin{cases} ED[i-1][j-1] & \text{if } word1[i] = word2[j] \\ \min \begin{cases} ED[i-1][j] + 1 \uparrow D \\ ED[i][j-1] + 1 \leftarrow I \\ ED[i-1][j-1] + 1 \nwarrow R \end{cases} & \text{otherwise} \end{cases}$$

		A	R	T	S
M	0	1	2	3	4
A	2	1	2	3	4
T	3	2	2	2	3
H	4	3	3	3	3
S	5	4	4	4	3

Time: $O(N \times M)$

Space: $O(N \times M)$

Backtracking: $\rightarrow$ Diagonal değerler eşitse rast aynı
Eşit değilse 3 operatör
yönleri kırıstanmalı

S
S
H S
S
T H S
T S

MATHS

- ARTS

MATHS
A R T S

MA-THS

-ART-S

for i = 1 to N
for j = 1 to M
ED[i][j], ...

Longest Common Subsequence

Subsequence:

yakamoz → yak, ya2, yaka, a2, ka2

LCS is NOT Unique

word 1

yaka

word 2

yormak

"yak" ↔ "yak"

common subsequence

$O(n \times 2^m)$



word 1 ↓ word 2 →
LCS[i][j]

case 1: word 1[i] = word 2[j]; $LCS[i][j] = LCS[i-1][j-1] + 1$

$LCS(\text{word 1}[0...i], \text{word 2}[0...j])$

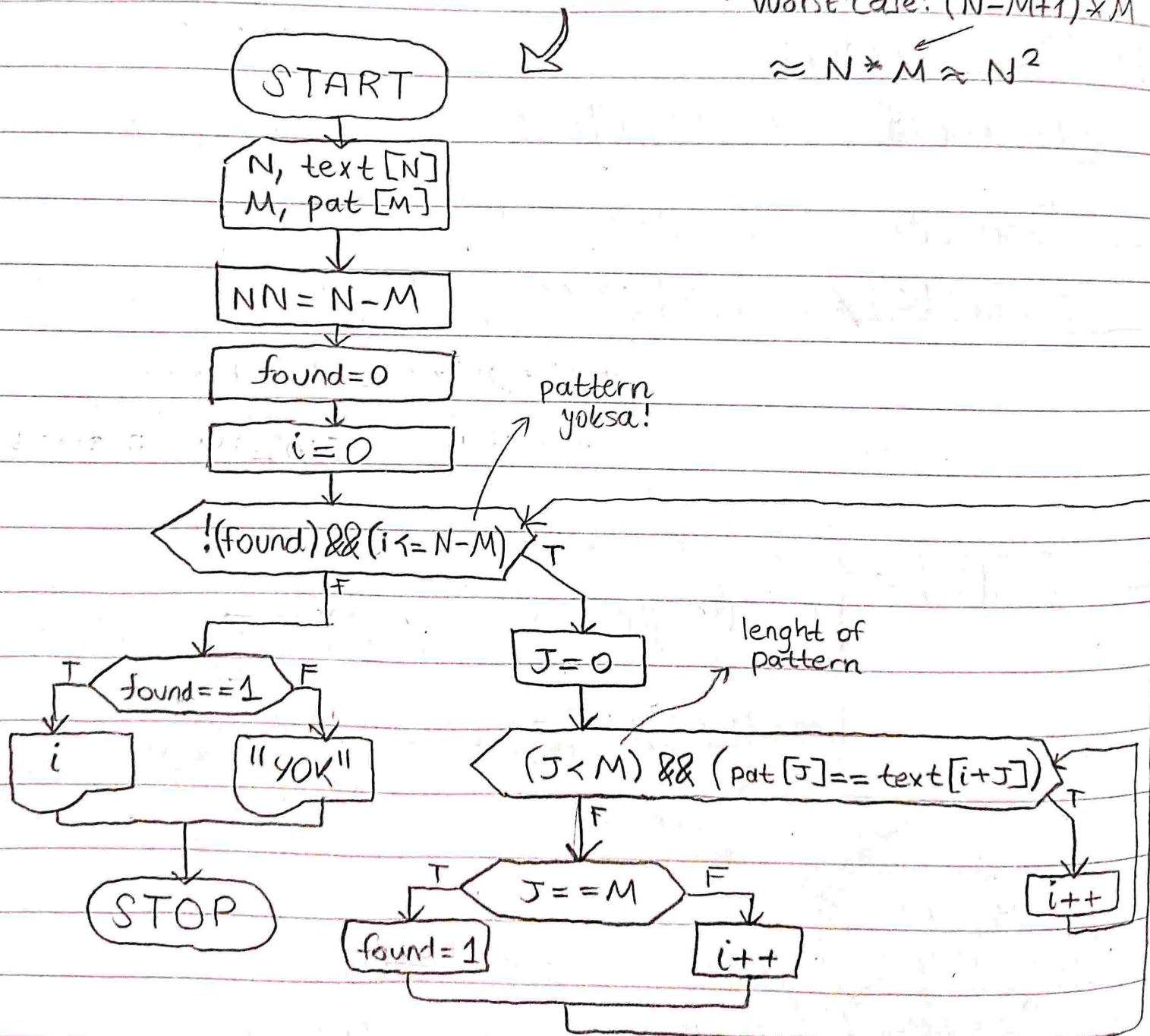
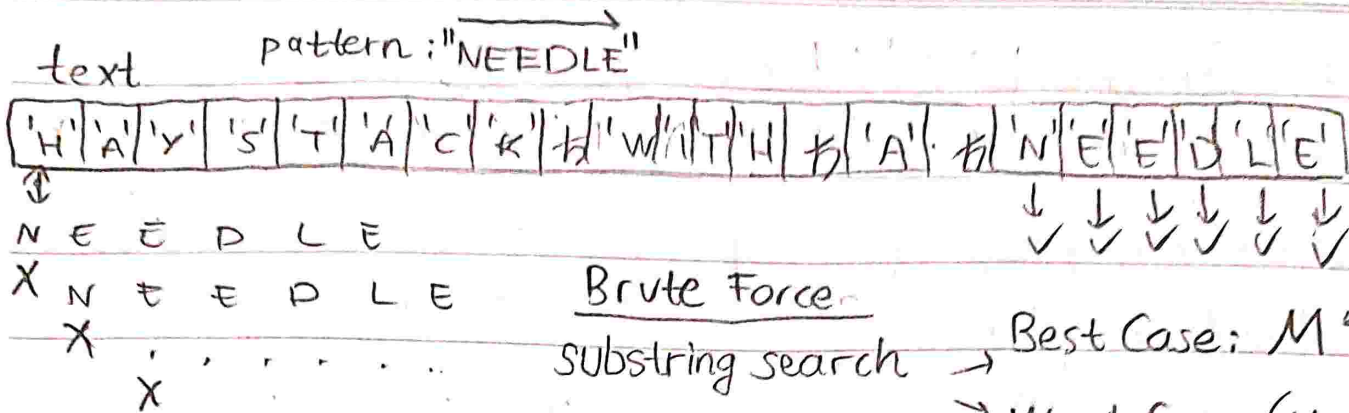
case 2: word [i] ≠ word [j] {
→ word 1[i] does not exist in subsequence
 $LCS[i][j] = LCS[i-1][j]$
→ word 2[j] does not exist in subsequence
 $LCS[i][j] = LCS[i][j-1]$

$$LCS[i][j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \text{ (initialization)} \\ LCS[i-1][j-1] + 1 & \text{if } i>0, j>0, \text{word 1}[i] = \text{word 2}[j] \\ \max \left\{ \underset{\substack{\downarrow \\ \text{word 1}[i] \\ \text{doesn't exist}}}{LCS[i-1][j]}, \underset{\substack{\downarrow \\ \text{word 2}[j] \\ \text{doesn't exist}}}{LCS[i][j-1]} \right\} & \text{if } i>0, j>0, \text{word 1}[i] \neq \text{word 2}[j] \end{cases}$$

B D C B

		0	0	0	0	0		"B"	"BDCB"	LCS: 1
B	0	1	1	1	1	1		"BA"	"BDCB"	LCS: 1
A	0	1	1	1	1	1		"BAC"	"BDCB"	LCS: 2
C	0	1	1	2	2	2		"BACD"	"BDCB"	LCS: 2 BD/BC
D	0	1	2	2	2	2		"BAKDB"	"BDCB"	LCS: 3 BCB BDB
B	0	1	2	2	3	3				

String Search Algorithms



String Search Algorithms

pattern : "NEEDLE"

F I N D I N A H A Y S T A C K N E E D L E text

N E E D L E
N E E D L E

S? N E E D
x →

L E
x ← ✓

N E E D L E
✓ ✓ ✓ ✓ ✓ ✓

BOYER - MOORE
Algoritması

sondan başa
doğru karşılaştırır

How much to skip?

Case 1: mismatch character is NOT in the pattern

NAHAY S
NEEDLE
x x x x x x

pattern[J]

skip = J + 1 //

Case 2: mismatch character is in the pattern

FIND I N
NEEDLE
x x x x x

skip = J - right Most Occurrence In Pattern

```
int boyer Moore ( char pat[], char text[])
{
    int i, j, skip=0, N= strlen(text), M= strlen(pat);
    for (i=0; i<=N-M; i+=skip)
    {
        j=M-1;
        while ((j>=0) && (pat[j]==text[i+j]))
        {
            j--;
        }
        if (j>=0) // pattern bulunamadı !
        {
            skip= j - table[text[i+j]];
            if (skip<0)
                skip= 1;
        }
        else return i; // index in text
    }
    return -1; // not found
}
```

Sınavlarda çok severiz bu konuyu - hoca

String Search Algorithms

NEEDLE	'A'	'B'	'C'	'D'	'E'	...	'L'	'N'	...	'Z'
0 1 2 3 4 5	-1	-1	-1	3	5	...	4	...	...	-1

↑
pattern'in kağıncı karakteri

```
void shiftTABLE(char pattern[], int table, int size)
{
    // tablo uzunluğu
    int i, M = strlen(pattern);

    for (i = 0; i < size; i++)
        table[i] = -1;

    for (i = 0; i < M; i++)
    {
        table[pattern[i] - 'A'] = i;
    }
}
```

BAYER MOORE → Average: N/M
→ Worst: $N * M$

UYGULAMA 2

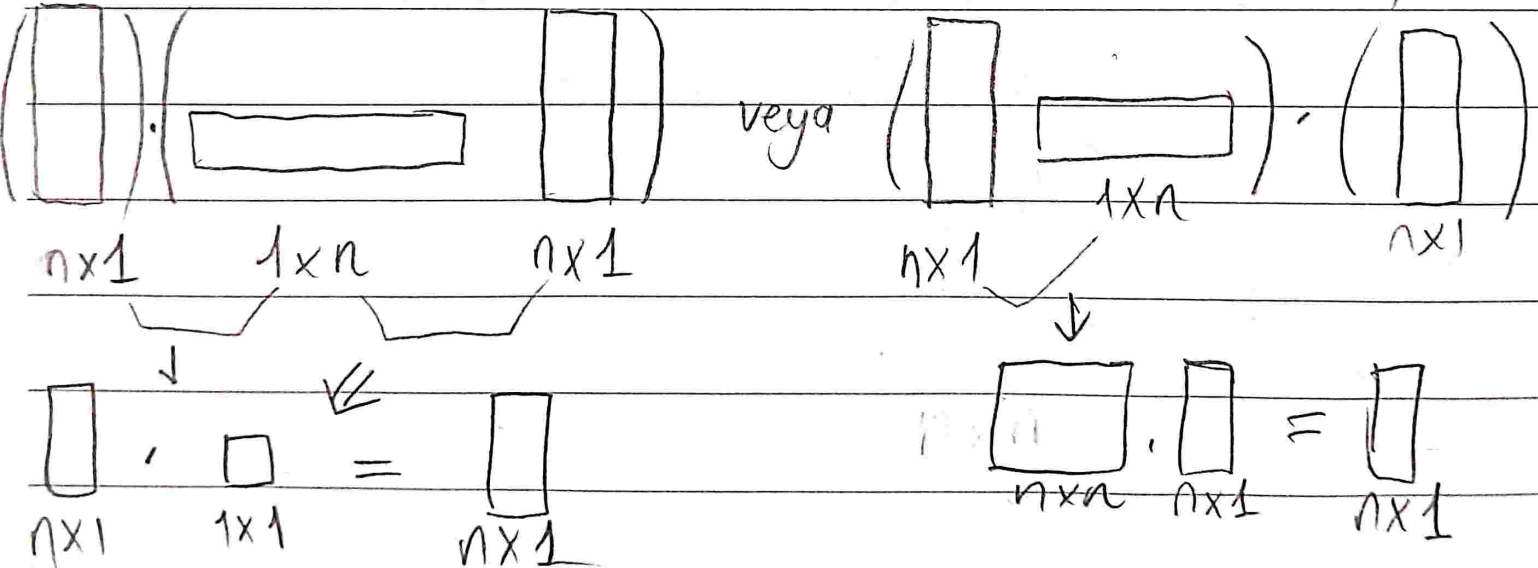
UYGULAMA

Dinamik Programlama

Matrix Chain Multiplication

$A_1 A_2 A_3 A_4 \dots A_n$

en az maliyetle tüm matrisleri nasıl çarparsınız?



A_1 A_2 A_3 10×30 30×5 5×60 $(A_1 A_2) A_3$

veya

 $(A_1)(A_2 A_3)$ $(10 \cdot 30 \cdot 5) = 1500$ çarpma işlemi 30×60 $30 \cdot 5 \cdot 60 = 9000$ $10 \cdot 30 \cdot 60 = 18000$ 10×5 5×60 $10 \cdot 5 \cdot 60 = 3000$ çarpma işlemi

27000 çarpma maliyeti

4500 çarpma maliyeti

örneğin $n=5$ için $\frac{1}{n} \left(\frac{2(n-1)}{n-1} \right) = 14$ farklı çarpım şeklimiz olur.

$n=10$ için 16796 farklı şekil var, dinamik programlama önemli.

Matrix Chain Multiplication

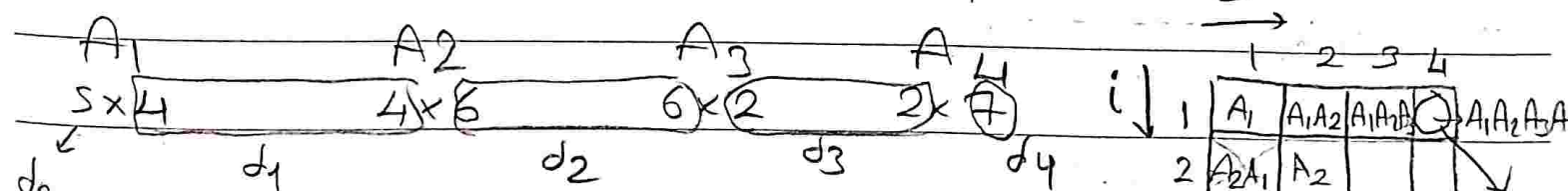


Table M (Cost Matrix):

i \ j	1	2	3	4
1	0	120	88	
2	X	0	48	
3	X	X	0	84
4	X	X	X	0

 $(A_1 A_2)_5$

$M[1][2] = 5 \times 4 \times 6 = 120$

$M[2][3] = 4 \times 6 \times 2 = 48$

$M[3][4] = 6 \times 2 \times 7 = 84$

$$(A_1 A_2 A_3) \quad (A_1 A_2)(A_3) \rightarrow M[1][2] + M[3][3] + 5 \cdot 6 \cdot 2 = 180$$

$$M[1][1] + M[2][3] + 5 \times 4 \times 2 = 88$$

$(A_1 A_2 A_3)$

MIN = 88

$M[1][3] = 88$

Parantez konumu nerede?

Table S (Split Matrix):

k \ j	1	2	3	4
1	0	1	1	
2		0	2	
3			0	3
4				0

$i \rightarrow$

M

0	120	88	158
X	0	48	104
X	X	0	84
X	X	X	0

A_2, A_3, A_4

$A_1 \quad A_2 \quad A_3 \quad A_4$
 $5 \times 4 \quad 4 \times 6 \quad 6 \times 2 \quad 2 \times 7$

$(A_2)(A_3, A_4)$

$(A_2 A_3)(A_4)$

$M[2][2] + M[3][4] + 4 \times 6 \times 7 = 252$

$M[2][3] + M[4][4] + 4 \times 2 \times 7 = 104$

MIN
104
 $k=3$

S
 (k)

0	1	1	3
X	0	2	3
X	X	0	3
X	X	X	0

a) $(A_1)(A_2 A_3 A_4)$

b) $(A_1 A_2)(A_3 A_4)$

c) $(A_1 A_2 A_3)(A_4)$

$M[1][4] = \min \begin{cases} \text{a) } M[1][1] + M[2][4] + 5 \times 4 \times 7 = 244 \\ \text{b) } M[1][2] + M[3][4] + 5 \times 6 \times 7 = 414 \\ \text{c) } M[1][3] + M[4][4] + 5 \times 2 \times 7 = 158 \end{cases} \Rightarrow 158$

$k \rightarrow \min \Rightarrow 3$

Matrix
Multiplication

$M[i][j] = M[i][k] + M[k+1][j] + (d_{i-1} \cdot d_k \cdot d_j)$

// cost

$S \rightarrow ((A_1) A_2 A_3) A_4$
 $k=3$
 $k=1$

Let M $n \times n$ matrix, d array dimensions

Let S $n \times n$ matrix // index

for $i=1$ to n do

$M[i][i] = 0$

for $L=1$ to $n-1$ do // $L = \# \text{ of matrix} - 1$
 $(n-1)$

for $i=1$ to $n-L$ do

$J = i+L$

for $k=i$ to $J-1$

$\text{cost} = M[i][k] + M[k+1][J] + (d[i-1] \cdot d[k] \cdot d[J])$

if ($\text{cost} < M[i][J]$)

$M[i][J] = \text{cost}$

$S[i][J] = k$

$(3 \times 2 \quad 2 \times 5 \quad 5 \times 3)$
 $3 \times 2 \times 3 \quad 18$
 $0 \quad 30 \quad 15 + 60, 2 \quad 6$
 $0 \quad 902$
 0
 244

ACTIVITY SELECTION (/WEIGHTED INTERVAL SCHEDULING)

N jobs $\rightarrow$ weight
 $\rightarrow$ start time - finish time } max profit



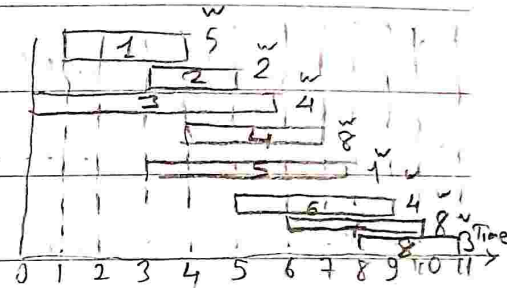
Örnek $\rightarrow$ 6 jobs

Job	1	2	3	4	5	6
start:	1	3	0	5	8	5
finish:	2	4	6	7	9	9
Weight	5	2	4	8	1	4

✓ 1, 2, 4, 5 ✗ 3, 6

(gelişmeyen en fazla sayıda işi seçme problemi)

$\rightarrow$ better to sort jobs by finished time



J	w(J)	q(J)	MP $\rightarrow$ max profit
0	-	-	0
1	5	0	$5 \rightarrow \max(MP[0], w(1) + MP[q_1]) = 5$
2	2	0	$5 \rightarrow \max(MP[1], w(2) + MP[q_2]) = 5$
3	4	0	$5 \rightarrow \max(MP[2], w(3) + MP[q_3]) = 5$
4	8	1	$13 \rightarrow \max(MP[3], w(4) + MP[q_4]) = 13$
5	1	0	13
6	4	2	13
7	8	3	13
8	3	5	$16 \rightarrow \max(MP[7], w(8) + MP[q_8]) = 16$

$$\text{Max_Profit} = \begin{cases} 0 & , J=0 \\ \max(MP[J-1], w(J) + MP[q_J]) & , \text{otherwise} \end{cases}$$

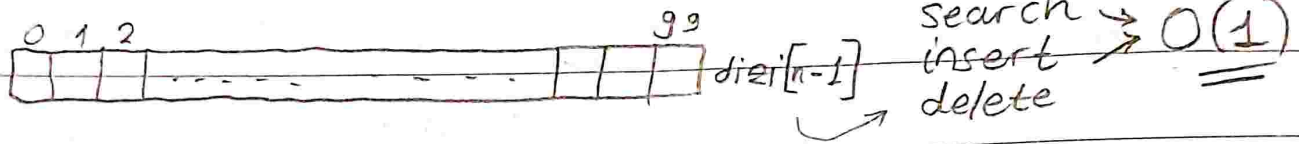
find_solution(J):
 if (J == 0)
 return 0
 else if $w(J) + MP[q_J] > MP[J-1]$
 return findSolution(q_J)
 else
 return findSolution(J-1)

$MP[0] = 0$
 for J=1 to N do
 $MP[J] = \max \dots$

Hashing

Worst Case $\rightarrow$ Linear Search Binary Search BST Balanced BST
 Searching n $\log_2 n$ n $\log_2 n$
 Sorting

1-100 arasında öğrenci numarası aransın:



1-1.000.000 arası numara

$\downarrow$
 100 $\leftarrow$ number of objects

1.000.000 array size??

Hashing
 Searching $O(1)$

~ Hashing ~

(for IBM-701 computers $\rightarrow$ 1953)

20011503

öğrenci numarası

hash function

index

Search
insert
delete

index

hashtable

20011503

$\rightarrow$ farklı numaralar için aynı indiş verilirse? $\rightarrow$ collusion problem

Hash Function

must be $\rightarrow$ easy to compute
 $\rightarrow$ consistent

Division Method

Multiplication Method

(en çok kullanılan yöntemler)

(1) Division Method

$$\text{hashfunction}(\text{key}) = \text{key} \% m$$

example: 23

hash table size

key: 23, table size: 11

$$\text{index} = 23 \bmod 11 = 1$$

0	
1	23
2	
3	
4	
5	
6	
7	
8	
9	
10	

Hashing

② Multiplication Method

Knuth

$$\text{hashFunction}(\text{key}) = \lfloor m * [(\text{key} * A) \bmod 1] \rfloor$$

$$A = \frac{\sqrt{5}-1}{2} = 0.618033$$

table size constant

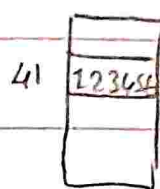
$$m = 10,000 \quad \text{key} = 123456$$

$$\text{hashFunction}(123456) = \lfloor 10000 * (123456 * 0.618033) \bmod 1 \rfloor$$

$$76300.004151$$

$$10000 * 0.004151$$

$$\lfloor 41.151 \rfloor = 41$$



hashtable

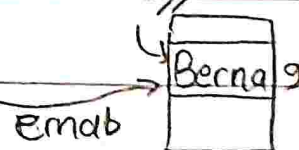
→ Key must be unique → TCKN, student Id, phone Nmbr...

If not ~,

① String "berna" $\text{hashF}(\text{"berna"}) \rightarrow \text{integer Index}$

$$'b' - 'a' + 'e' - 'a' + \dots = 42 \quad \% \quad m^{11} = 9$$

$$\text{"ernab"} = 42$$



Gakışma!

Horner's Method

$$R=31 \quad L = \text{strlen}(\text{"berna"})$$
$$31^{L-1} * \text{str}[0] + 31^{L-2} * \text{str}[1] + \dots + 31^0 * \text{str}[L-1]$$

(Bir asal sayı kullanılarak her harfe farklı ağırlık verilir.)

```
length = strlen(str);
key = 0;
for (i = 0; i < length; i++)
{
    tmp = str[i] - 'A' + 1;
    key = (31 * key + tmp);
}
```

prime number

JAVA:
Binary representation
of a string

Hashing

② Compound Key

day-month-year $\rightarrow$ key = $\left[(\text{day} \times \overset{\text{prime integer}}{R} + \text{month}) \% m + R \times \text{year} \right] \% m$

③ Floating Point

key = 35.780 $\rightarrow$ round $\left[\text{key} \times \overset{\text{constant}}{10000} \right] \% m$

Çakışma problemi nasıl çözülür

a) Open Addressing

a.1) Linear Probing

Use an array of size $M \gg N$ $\rightarrow$ table size $\rightarrow$ number of elements
Yer doluyorsa (linear şekilde) boş yer aranır
prime number

$$\text{hashFunction}(\text{key}, i) = \left[\underbrace{\text{hashFunction}(\text{key})}_{\text{key \% } m} + i \right] \bmod m$$

$\downarrow$
step size (check)

$m = 7$

0	NULL	19
1	NULL	
2	NULL	
3	NULL	
4	NULL	
5	NULL	5
6	NULL	12

key = 5 $\rightarrow$ adr: $5 \% 7 = 5$

key = 12 $\rightarrow$ adr: $12 \% 7 = 5 \rightarrow$ DOLU

$\rightarrow$ key = $(12 \% 7 + i) \% 7 = 6$

$\downarrow$
1

key = 19 $\rightarrow 19 \% 7 = 5 \rightarrow$ DOLU $i = 1 \rightarrow$ DOLU

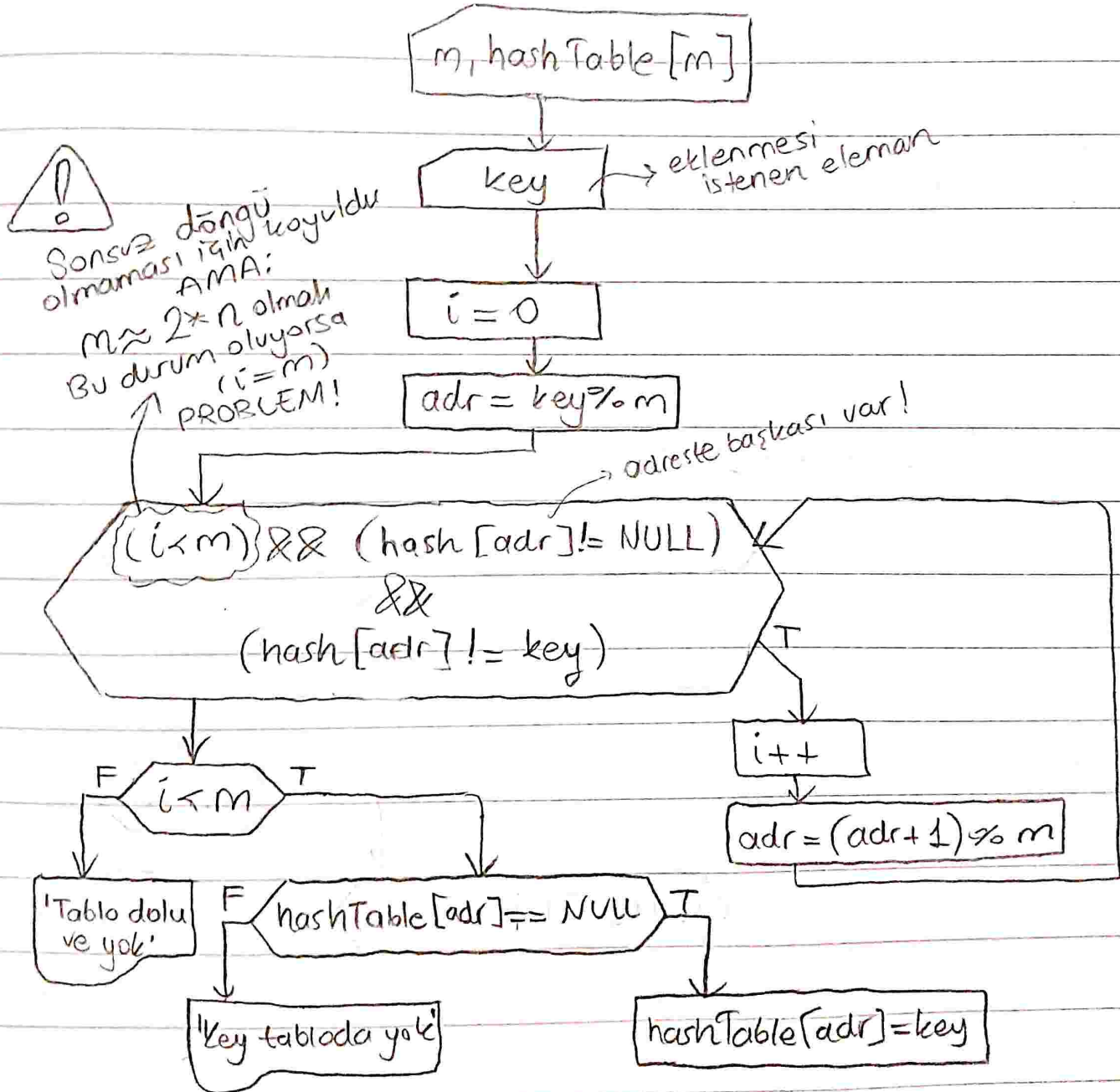
$\rightarrow$ key = $(19 \% 7 + 2) \% 7 = 0$
 $i = 2$

$\rightarrow$ M genelde 2 kat seçilir.

$\rightarrow$ key = 0 gelse? $\rightarrow$ CLUSTERING PROBLEM

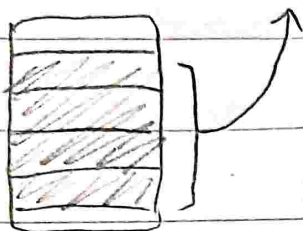
(Linear Probing)

Bir hash tablosuna Linear Probing ile yeni eleman eklenecek. Eleman daha önce ekliyse kullanıcıya uyarı mesajı veren, yoksa tabloya yerleştiren algoritmayı tasarlayınız.



Hashing

Clustering Problem



Çözüm

~~Linear Probing~~ → Double Hashing

a.2) Double Hashing

(Table size = $m = 13$)

$$\text{hashFunction}(\text{key}, i) = [\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})] \% m$$

↓
 i^{th} step
(check)

↓
 $\text{key} \% 13$
↑
 m

↓
 $\text{key} \% 11$

genellikle m 'den küçük,
kaç adım ileri gidilecek

0	
1	14
2	
3	
4	
5	
6	27
7	
8	40
9	
10	
11	12
12	

key = 27

$$\text{hash1}(\text{key}) = 27 \% 13 = 1 //$$

$i = 1$

$$1 + 1 * \text{hash2}(\text{key}) = 6$$

$$\uparrow \quad \quad \quad \underbrace{27 \% 11}_{5} \quad \quad \quad 6 \% 13 = 6 //$$

hash1(key)

key = 40

$$40 \% 13 = 1 \rightarrow \text{dolu}$$

$$(1 + 1 * (\underbrace{40 \% 11}_{7})) \% 13 \Rightarrow 8 //$$

Double Hashing ile peş peşe birikmenin önüne geçilmiş olunur.

Hashing

Peki nasıl $O(1)$?

$$\text{Load Factor } \alpha = \frac{N}{M}$$

→ number of elements

→ table size

$$\alpha \approx 1/2$$

Linear Probing

	%.50	%.66	%.75	%.90	⇒ (doluluk oranı)
search →	1.5	2.0	3.0	5.5	
insert →	2.5	5.0	8.5	55.5	

Double Hashing

search →	1.4	1.6	1.8	2.6
insert →	1.5	2.0	3.0	5.5

Birthday Paradox

Birthday Paradox

Uniform Hashing Assumptions

Table size: m key = $m, 2m, 3m, \dots$

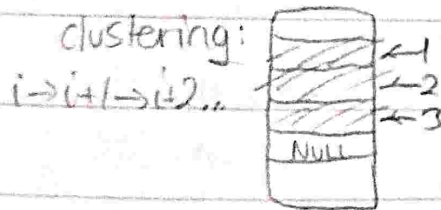
Each key is equally likely to hash to any slot in the table

$$\Pr[\text{hashfunction}(\text{key}) = i] = \frac{1}{m}, \text{uzun vadede ağırlıklı ortalama}$$

$$\text{Expected value: } E[x] = \sum_{i=1}^n i \times \text{Probability}(x=i)$$

$$\text{Zar problemi: } E[x] = \sum_{i=1}^6 i \times \underbrace{\Pr[i]}_{1/6} = 3.5$$

Linear Probe



probability of a slot

full α

empty $1-\alpha$

probability of a second slot is empty: $\alpha \cdot (1-\alpha)$

k th slot is empty: $\alpha^{k-1} (1-\alpha)$

ne yapmalıyız ki karmaşıklık $O(1)$ olsun?

Load Factor $\rightarrow \alpha = \frac{n}{m}$ $n \rightarrow$ # of elements
 $m \rightarrow$ table size

Hashing

$$P(k \text{ full cells before empty:}) = \alpha^k \times (1-\alpha) \quad \rightarrow (k+1)^{\text{th}} \text{ empty}$$

Geometrik dağılım:

$$\text{Expected value: } E[k \text{ full cells}] = \frac{1-p}{p} = \frac{1-(1-\alpha)}{1-\alpha} = \frac{\alpha}{1-\alpha}$$

$$E[\text{full}] = \frac{1}{p} = \frac{1}{1-\alpha}$$

Insert için toplam başarılı probe sayısı

Unsuccessful search

$$\text{Search: } E[\text{successful search}] = \frac{1}{2} \left[1 + \frac{1}{1-\alpha} \right] = \frac{1}{2} \left[\frac{2-\alpha}{1-\alpha} \right]$$

yaklaşık olarak cluster'in ortasını yalıtılır

Örnek: Doluluk oranı $1/2$ ise $\rightarrow \alpha = \frac{n}{m} = \frac{5}{10} = 0.5 \quad \frac{3}{2} = 1.5$

olasılıksal olarak 1.5 adım sonra aranan eleman bulunacak

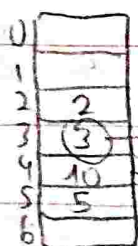
olasılıksal olarak 2 adım sonra insert edilecek

(burayı tam yazamadım, hoca işlemlerde bir hata var gibi bir şey demişti sanırım)

	worst case			average case		
	Search	Insert	Delete	Search	Insert	Delete
Sorted Array	$\log N$	N	N	$\log N$	$N/2$	$N/2$
Unsorted Array	N	N	N	$N/2$	$N/2$	$N/2$
Linear Probing	N	N	N	1	1	1

Universal Hashing Assumption

Deletion Problem



Silinen yerleri işaretlessek?

DELETE?

3 silindi $\rightarrow$ NULL

10'u ararsak $\rightarrow 10 \% 7 = 3 \rightarrow$ NULL $\rightarrow$ 10 yok!?

* table size should be usually prime

$$n \rightarrow 5 \quad m = 11$$

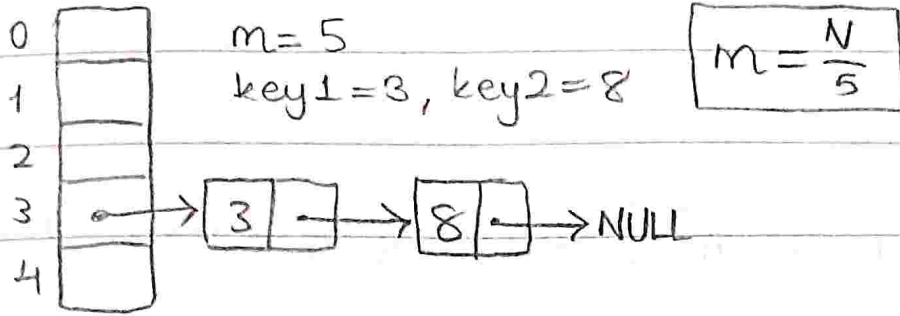
$$n \rightarrow 10 \quad m = 23$$

neden? $\rightarrow$

m: 8 $\rightarrow$ keys 64, 400, 128...

Hashing

Separate Chaining



```
struct Node
```

```
{
```

```
    int key;
```

```
    char name[40];
```

```
    struct Node *next;
```

```
}
```

```
struct Hash
```

```
{
```

```
    struct Node *head;
```

```
    int count;
```

```
}
```

```
struct hash *hashtable = NULL;
```

```
void inserttoHash table (int key, char name)
```

```
{
```

```
    int hashIndex;
```

```
    hashIndex = key % M;
```

```
    struct node *newNode = createNode (key, name);
```

```
    if (!hashTable[hashIndex].head)
```

```
    {
```

```
        hashtable[hashIndex].head = newNode;
```

```
        hashtable[hashIndex].count = 1;
```

```
    }
```

```
    else
```

```
    {
```

```
        newNode->next = hashtable[hashIndex].head;
```

```
        hashtable[hashIndex].head = newNode;
```

```
        hashtable[hashIndex].count++;
```

```
    }
```

```
}
```

Hashing

```
int main ()
{
    int key;
    char name;
    ~
    hashTable = (struct hash*) calloc (M, sizeof(struct Hash));
    ~
    insertto HashTable (key, name);
    ~
}
```

$M \gg N \rightarrow$ çok boş ↓ delete için iyi

	worst			average		
	Search	Insert	Delete	S	I	D
Separate Chaining	N	1	N	1	1	1

Universal Hashing

Bad keys $\rightarrow$ Bad hashing

key $\rightarrow m, 2m, 3m$

$h_1(), h_2(), \dots$

randomly chosen

Hashing Application

	key ↓ (word)
spell checker	(word)
browser	(URL)
Spam filter	(IP)
credit cards	(card number)

p : large prime $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$

$$h_{a,b}(\text{key}) = [(a * \text{key} + b) \% p] \% m$$

table size is not random!

Example: $p=17, m=7, a=3, b=4$

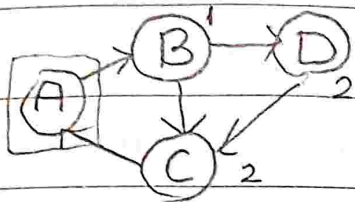
$$h_{3,4}(8) = ((\underbrace{3 * 8}_{28} + 4) \% 17) \% 7 = 4$$

$\swarrow \quad \searrow$
 $a \quad b$

Shortest Path

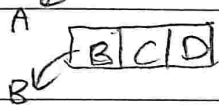
Shortest Path

BFS



A	B	C	D
0	∞	∞	∞
	1	2	2

Queue [A|B]



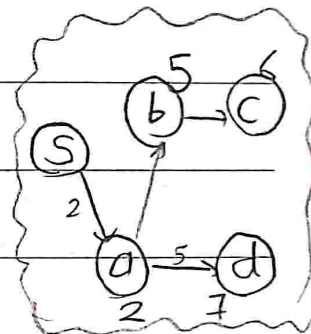
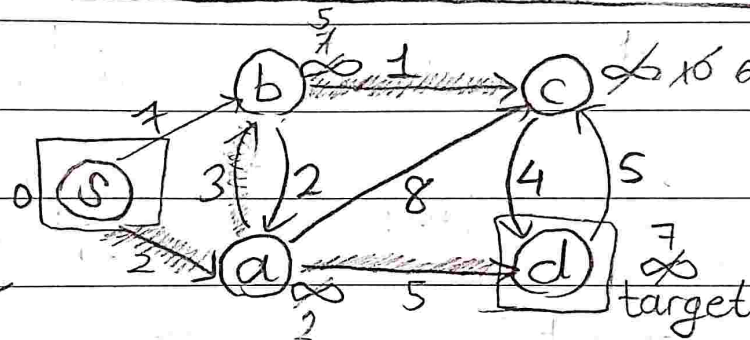
(unweighted)
digraph

(weighted
di-graph
positive weights) ?

Source $\rightarrow$ target
SHORTEST PATH?

Greedy
Algorithm

Dijkstra Algorithm



① Initialization

set distance of source = 0

set distance of all others = ∞

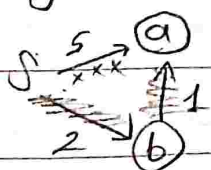
mark all nodes unvisited

② Select closest unvisited node

choose node u with minimum distance

mark u as visited

③ Edge Relation



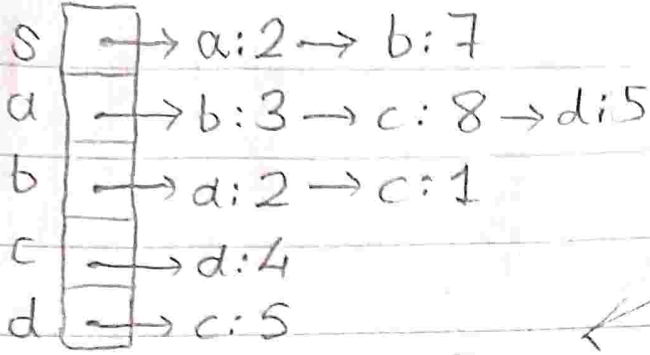
if $\text{dist}[u] + w(u, v) < \text{dist}[v]$

$\text{dist}[v] = \text{dist}[u] + w(u, v)$

④ REPEAT from 2

UNTIL TARGET NODE IS FINALIZED

Shortest Path: Dijkstra



Initialization

v	s	a	b	c	d
d[v]	0	∞	∞	∞	∞
pred[v]	NULL	NULL	NULL	NULL	NULL
color[v]	w	w	w	w	w

w: unvisited, b: visited

priority queue (Heap tree)

v	s	a	b	c	d
d[v]	0	∞	∞	∞	∞

dequeue

v	s	a	b	c	d
d[v]	0	2	7	∞	∞
pred[v]	N	S	S	N	N
color[v]	b	w	w	w	w

(her adımda en küçük değeri seçilir)

v	a	b	c	d
d[v]	2	7	∞	∞

dequeue

v	s	a	b	c	d
d[v]	0	2	5	10	7
pred[v]	N	S	a	a	a
color[v]	b	b	w	w	w

v	b	c	d
d[v]	5	10	7

dequeue

v	s	a	b	c	d
d[v]	0	2	5	6	7
pred[v]	N	S	a	b	a
color[v]	b	b	b	w	w

v	c	d
d[v]	6	7

dequeue

v	s	a	b	c	d
d[v]	0	2	5	6	7
pred[v]	N	S	a	b	a
color[v]	b	b	b	b	w

v	d
d[v]	7

dequeue

Shortest Path: s → a → d : 7

(d[v] ⇒ distance to source, pred[v] ⇒ v'ye en son nereden gelindiği, color[v] ⇒ düğümün durumu)

Shortest Path: Dijkstra

Dijkstra (G, s, w)

{
for (each $u \in V$)

{

$d[u] = \infty$

color[u] = white

}

$d[s] = 0$

pred[s] = NULL

PQ = (queue with all vertices)

while (non-empty(PQ))

{

$u = \text{extract-min}(PQ)$

for (each $v \in \text{Adj}[u]$)

{

if ($d[u] + w(u, v) < d[v]$)

{

$d[v] = d[u] + w[u, v]$

decrease-key(PQ, v, $d[v]$)

pred[v] = u

}

color[u] = black

}

}

extract-min $\Rightarrow$ heap ile: $O(\log V)$

decrease-key $\Rightarrow O(\log V)$; en fazla E kez

Average case: $O((V+E) \log V)$

Worst case: $O(V^2 \log V)$

$\rightarrow$ en küçük mesafeyi bulma

Shortest Path: Floyd - Warshall

Floyd-Warshall All Pair Shortest Path (APSP)

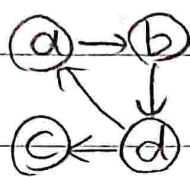
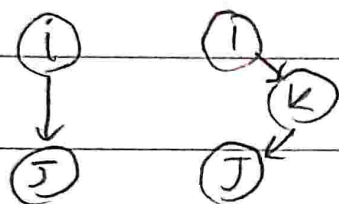
Finds shortest path between all pairs of vertices!

① Warshall Algorithm

Is there a path from i to j

USE TRANSITIVE CLOSURE {

$$R[i][j] = R[i][j] \vee (R[i][k] \wedge R[k][j])$$



$$A = (R_0) \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$T = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

transitive closure

$$R_k[i][j] = 1 \text{ if } \begin{cases} R_{k-1}[i][j] = 0 \\ \text{AND} \\ R_{k-1}[i][k] = 1 \\ \text{AND} \\ R_{k-1}[k][j] = 1 \\ \text{OR} \\ R_{k-1}[i][j] = 1 \end{cases}$$

$$R_0 = A$$

for $k=1$ to n do

for $i=1$ to n do

for $j=1$ to n do

$$R_k[i][j] = R_{k-1}[i][j] \text{ OR}$$

$$[R_{k-1}[i][k] \text{ AND } R_{k-1}[k][j]]$$

örnek:

$$R_1 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

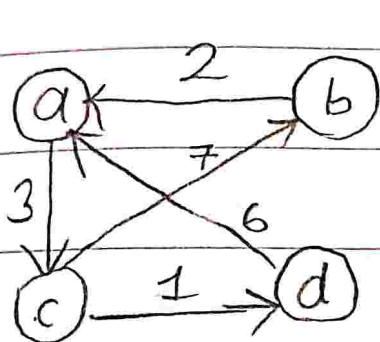
$d \rightarrow b: \begin{cases} d \rightarrow a: 1 \\ a \rightarrow b: 1 \end{cases} \Rightarrow 1$

Shortest Path: Floyd - Marshall

Floyd-Warshall APSP

DP $\rightarrow$ (Dinamik Programlama)

$$\text{distance}[i][j] = \min(\text{distance}[i][j], \text{distance}[i][k], \text{distance}[k][j])$$



$$\Rightarrow W = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$

$$D_0 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{pmatrix} \end{matrix}$$



$$D_1 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

$$D_1 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

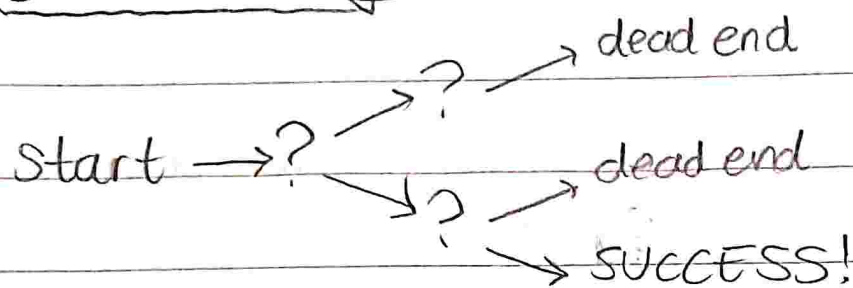


$$D_2 = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{pmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & 5 & \infty \\ 9 & 7 & 0 & 1 \\ 6 & \infty & 9 & 0 \end{pmatrix} \end{matrix}$$

...

Backtracking

Backtracking



① TRY

② CHECK

③ IF FAILS TRY ANOTHER OPTION!

N-QUEENS PROBLEM

Hıgbiri birbirine gakişmadan yerleştirme

① Brute Force → Check ALL possible solutions

$\binom{64}{8} = 4,426,165,368$ positions to try...

② One per row:

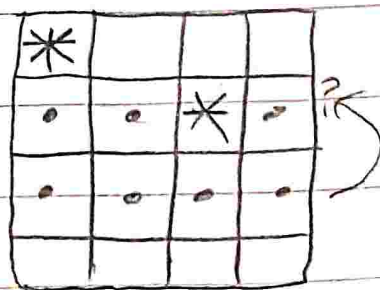
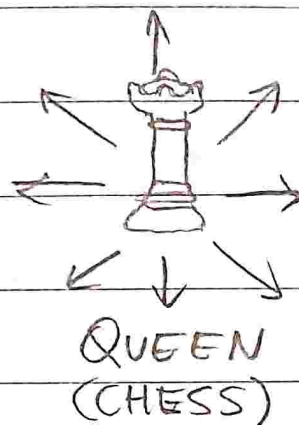
$8^8 = 16,777,216$ positions

③ One per row and per column

$8! = 40,320$ positions

④ BACKTRACKING

Only 2057 trials!



Brute Force

tries all possibilities

no structure

always exponential

(most of the time)

Backtracking

prunes invalid solutions early

recursive

constraint driven

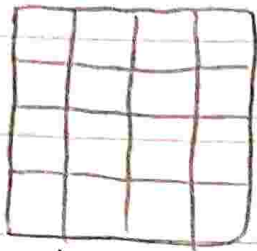
still exponential

but FASTER!

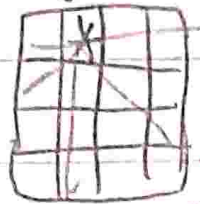
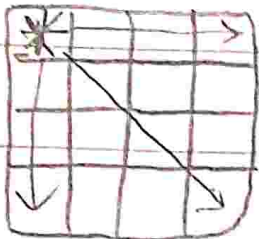
Backtracking

state space tree

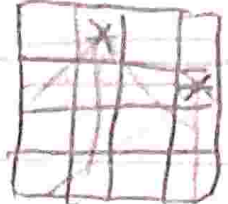
4 queen



1st row → 1✓X → 2*



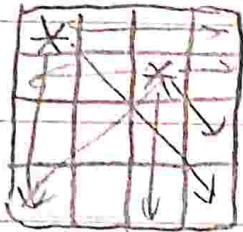
2nd: 1X 2X 3X 4✓



3rd: 1✓

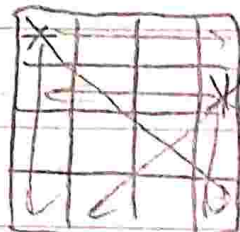
DEAD END! (BACKTRACK)

2nd row: 1* 2* 3✓X ← 4✓X ←



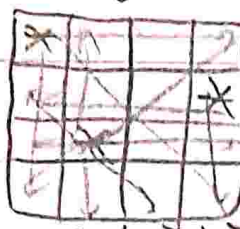
3rd row: 1* 2* 3* 4*

DEAD END!
(BACKTRACK)



1* 2* 3* 4*

DEAD END!
(BACKTRACK)



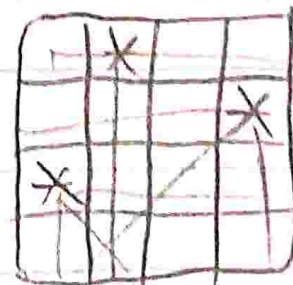
4th row: 1* 2* 3* 4*

DEAD END!
(BACKTRACK)

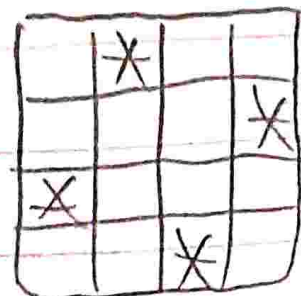
(Depth First Search)

Backtracking nasıl yapılabilir?

Stack, Recursion



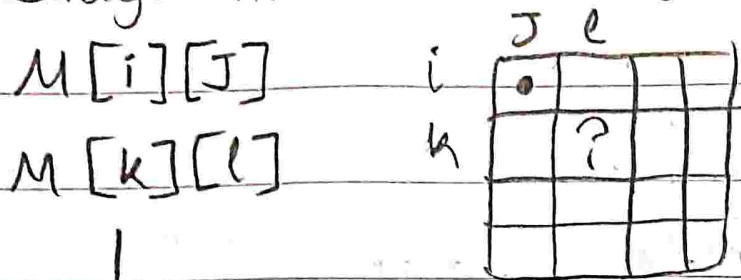
4th: 1* 2* 3✓



✓
SOLUTION

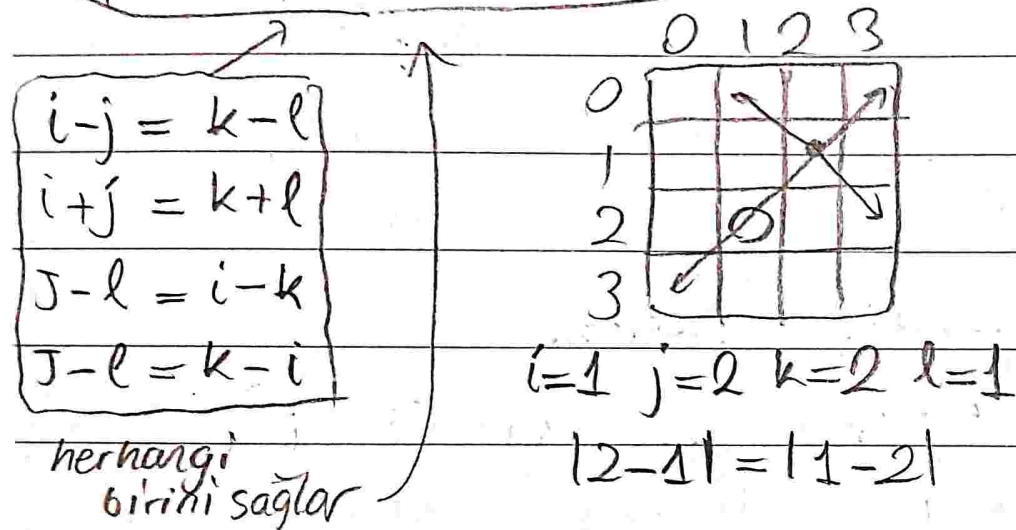
Backtracking

Diagonalleri basit kontrol etme yaklaşımı



they are on the same diagonal.

$$|j - l| = |i - k|$$



0...7 N-1

queen 0...N-1

1 3 0 2

(hangi sütunda!)

```
Backtrack (row)
{
    if (row == N)
        print queen
    end if
    for col = 0 to N-1 do
        if IsSafe (row, col) // boş mu?
            queen [row] = col
            Backtrack (row + 1)
            queen [row] = -1 // (undo choice)
        end if
    end for
END Backtrack
```

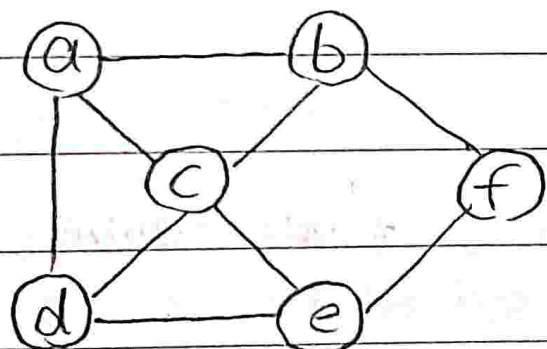
Backtracking

Is Safe (row, col)

```
{  
  for i = 0 to row-1 do  
    if queen[i] == col  
      return 0; // false!  
    if abs(queen[i] - col) == abs(i - row)  
      return 0;  
  end for  
  return 1; // yerleşebilir!  
}
```

Hamiltonian - Circuit Problem

her düğüme sadece 1 defa uğrayıp gezilebilir mi?



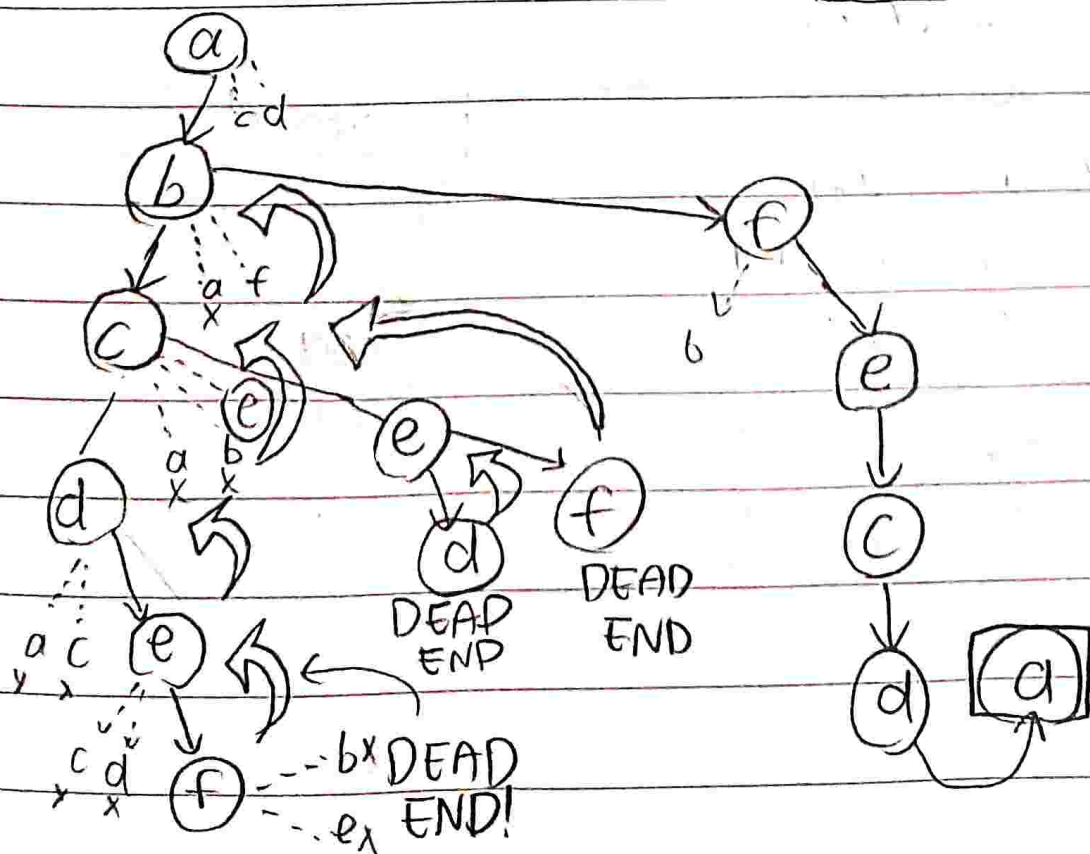
Graph(V, E)

visit every node
exactly ONE TIME!

also check:

3 coloring problem

state space tree

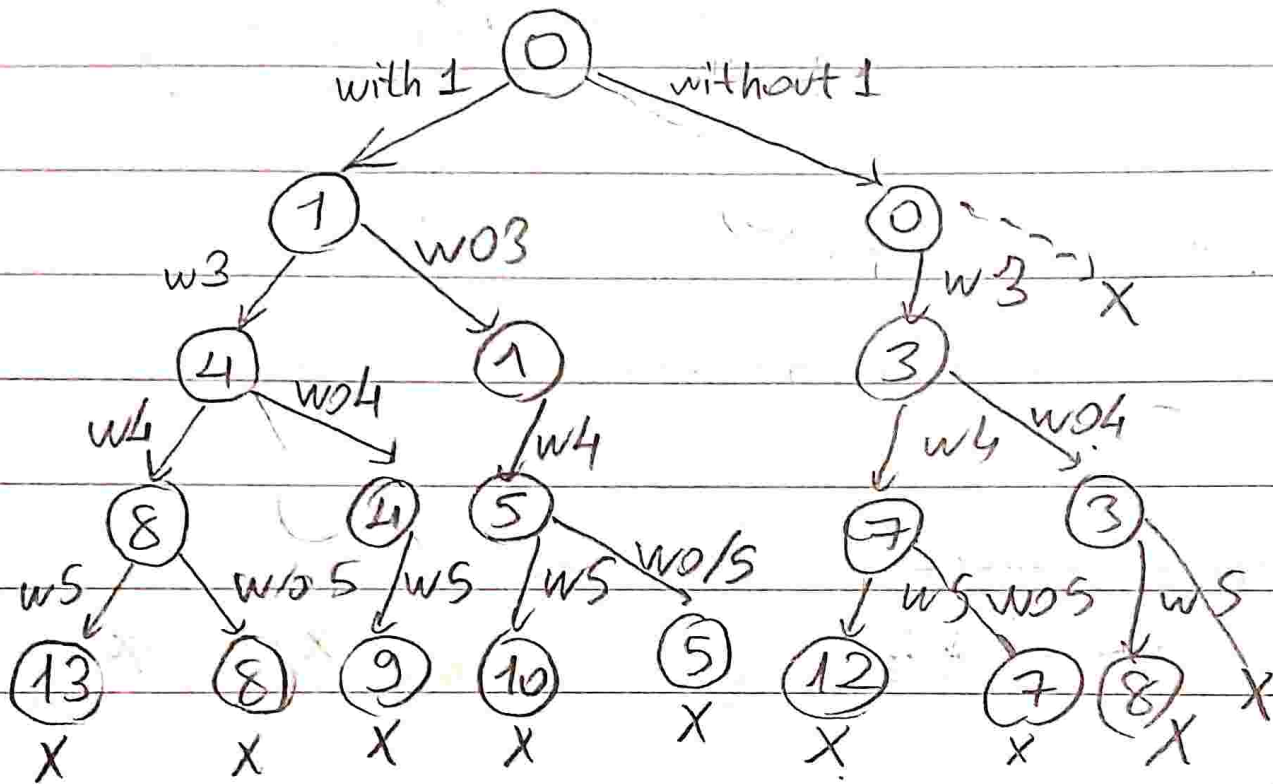


Backtracking

SUBSET SUM PROBLEM

$S = \{s_1, \dots, s_n\}$ positive integer
Ascending $\uparrow (d) \rightarrow$ can we get this?

$S = \{1, 3, 4, 5\}$ $d = 11$?



ASSIGNMENT PROBLEM

Bir işyerinde bir işe 4 kişi başvuruyor, ve her kişinin işler için istediği ücretler farklı. 4 kişi en az parayla nasıl yerleştirilir?

$N \rightarrow$ number of possible assignments : Brute : $N!$
Force : $N!$
worst case $\in O(n!)$

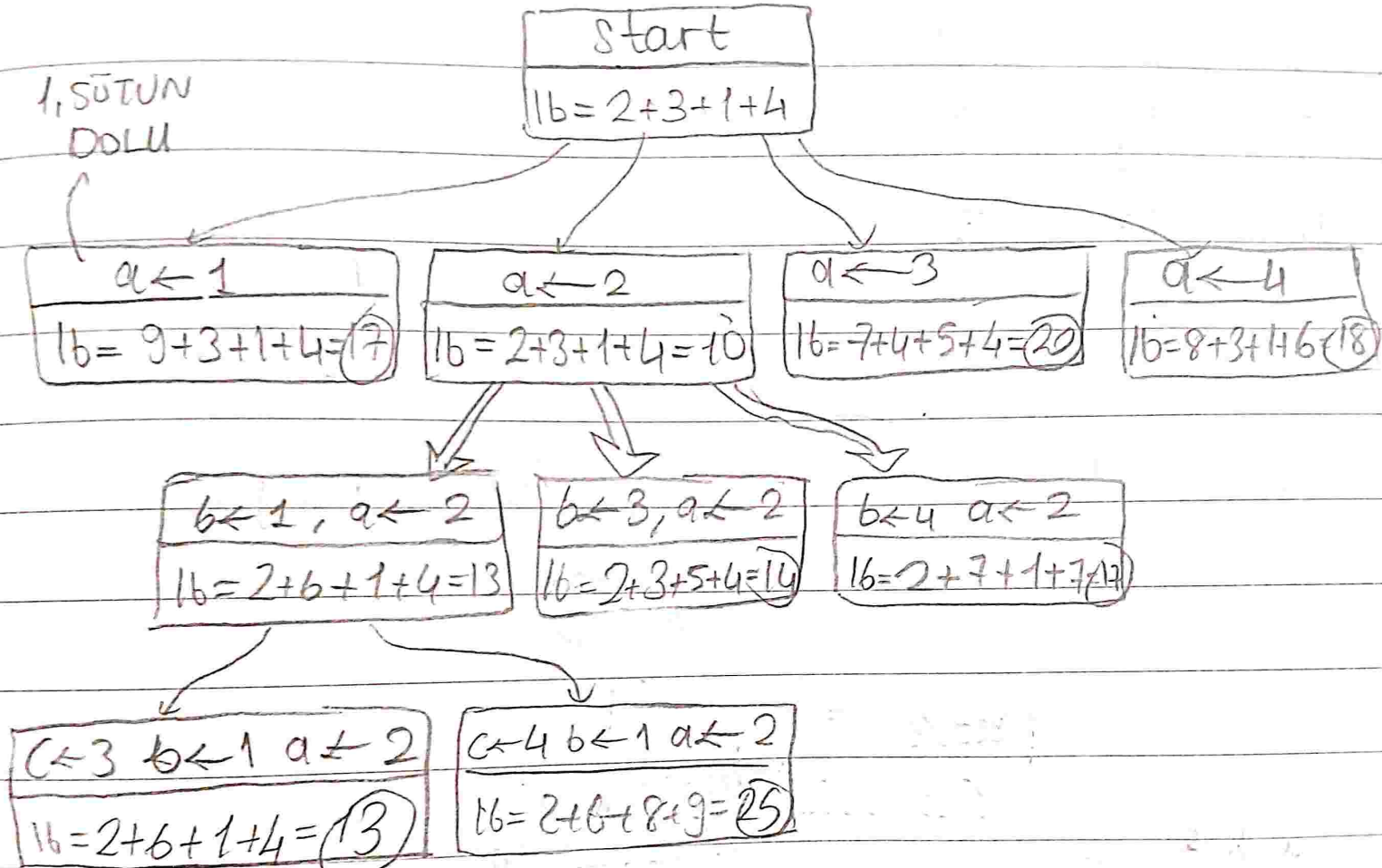
PRUNING may be much better

	J1	J2	J3	J4	
$C =$	9	*2	7	8	1
	6	4	*3	7	2
	5	8	*1	8	3
	7	6	9	*4	4

start
 $lb = 2 + 3 + 1 + 4 = 10$
lower bound
 $\swarrow$
sigmadı

Backtracking: Assignment Problem

state space tree



13'in altına inmek
İmkansız

SOLUTION

$a \leftarrow 2, b \leftarrow 1, c \leftarrow 3, d \leftarrow 4$

13

inebilseydik en düşük
ikinci değere BACKTRACKING

Backtracking

Knapsack Problem

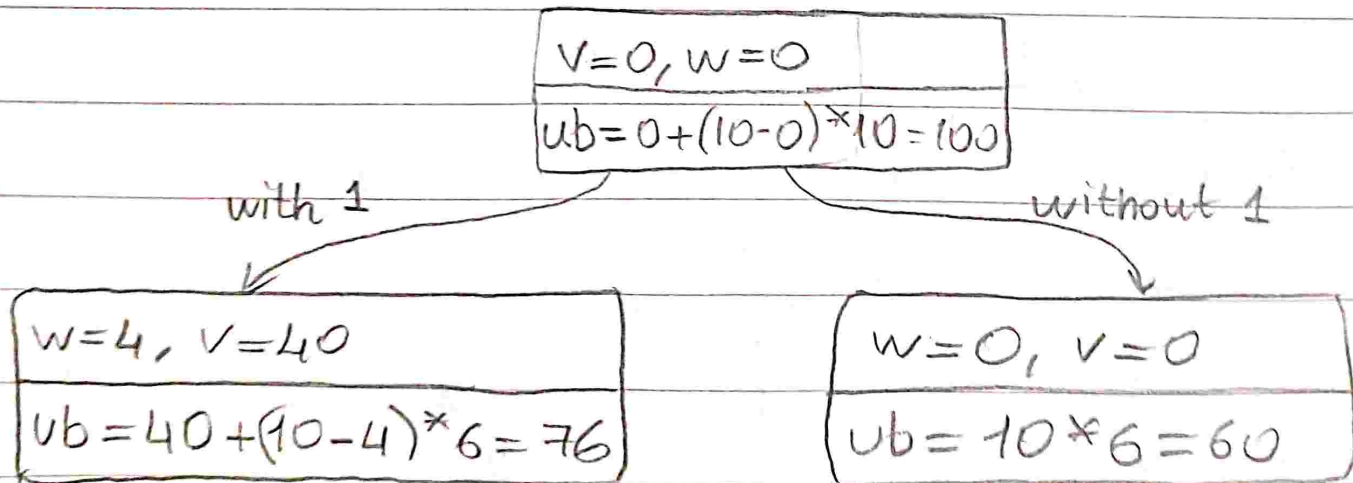
item	weight	value	v_i/w_i	birim kazanç
1	4	40	10	
2	7	42	6	
3	5	25	5	
4	3	12	4	

(sıralı)

$$W = 10 //$$

Hangi item'leri alacağını Branch and Bound ile gözün.

state space tree
of BRANCH AND BOUND



// kalanını kendiniz deneyin!

Back to Algorithm Analysis

polynomial $n^2, n^5, n^{10000} \rightarrow$ efficient

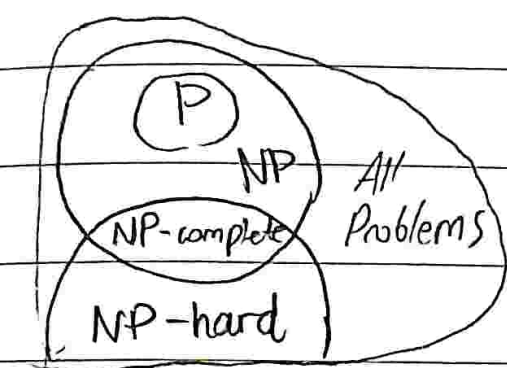
np: non deterministic polynomial

tractable problems can be solved in polynomial time.

$\hookrightarrow$ reverse: non-tractable/unsolvable

np np-hard np-complete

deterministic vs. non-deterministic algorithms



guessing stage

generates random candidate solution

verification stage

$\hookrightarrow$ polynomial part

(slaytta kalan her şey var)

Decision problemi mi?

$\hookrightarrow$ NP-complete olamaz!

$\hookrightarrow$ Evet

$\hookrightarrow$ Çözüm polinomsal zamanda doğrulanıyor mu?

$\hookrightarrow$ Hayır: NP değil

$\hookrightarrow$ Evet: NP

$\hookrightarrow$ NP-complete bir problem buna indirgenebiliyor mu?

Problem:

Optimizasyon: NP-hard

Decision: NP-complete

$\hookrightarrow$ Hayır: NP

$\hookrightarrow$ Evet: Complete-NP

Örn: Traveling Salesman $\rightarrow$ En kısa yol nedir? $\rightarrow$ Optimizasyon NP-hard

$\hookrightarrow$ toplam maliyeti $\leq K$ olan bir HC var mı? $\rightarrow$ Decision $\rightarrow$ NP